[ПРОВЕРИТЬ: название органа управления образованием]

[ПРОВЕРИТЬ: полное название колледжа]

КУРСОВОЙ ПРОЕКТ

по профессиональному модулю ПМ.07 «Разработка мобильных приложений»

на тему:

«Создание приложения для автоматического формирования персонального плана обучения»

Выполнил: студент 3 курса, группа [ПРОВЕРИТЬ]

[ПРОВЕРИТЬ: фамилия и инициалы студента]

Руководитель: [ПРОВЕРИТЬ: фамилия и инициалы]

Оценка: _____

[ПРОВЕРИТЬ: город, год]

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....4
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ПРИЛОЖЕНИЯ ДЛЯ ФОРМИРОВАНИЯ ПЕРСОНАЛЬНОГО ПЛАНА ОБУЧЕНИЯ.....8
1.1 Анализ предметной области.....8
1.2 Особенности мобильных приложений.....11
1.3 Принципы разработки мобильных приложений.....13
1.4 Анализ аналогичных решений.....14
1.5 Обоснование выбора технологий.....16
Выводы по главе 1.....22
ГЛАВА 2. РАЗРАБОТКА ПРИЛОЖЕНИЯ «МАРШРУТ».....24
2.1 Требования к приложению.....24
2.2 Архитектура приложения.....25
2.3 Проектирование базы данных.....28
2.4 Алгоритм формирования плана.....31
2.5 Интерфейс приложения.....36
2.6 Реализация основных модулей.....46
2.7 Тестирование.....49
2.8 Сборка и развёртывание.....50
Выводы по главе 2.....51
ЗАКЛЮЧЕНИЕ.....53
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....56
Книги и статьи.....56
Электронные ресурсы.....57
ПРИЛОЖЕНИЕ А. ТОПОЛОГИЧЕСКАЯ СОРТИРОВКА ТЕМ (АЛГОРИТМ КАНА).....61
ПРИЛОЖЕНИЕ Б. ХЭШИРОВАНИЕ ПАРОЛЕЙ PBKDF2-HMAC-SHA256 63
ПРИЛОЖЕНИЕ В. РАСКЛАДКА ТЕМ ПО ДНЯМ И ИНТЕРВАЛЬНЫЕ ПОВТОРЕНИЯ.....66

ПРИЛОЖЕНИЕ Г. ГЕНЕРАТОР ПЛАНА И ПРОВЕРКА ДЕДЛАЙНА.....69
ПРИЛОЖЕНИЕ Д. ПЕРЕПЛАНИРОВАНИЕ.....71
ПРИЛОЖЕНИЕ Е. ТАБЛИЦЫ БАЗЫ ДАННЫХ.....73
ПРИЛОЖЕНИЕ Ж. ПРАВИЛО ДОСТУПА К ЭКРАНАМ И МАРШРУТЫ. .76
ПРИЛОЖЕНИЕ И. МОДУЛЬНЫЕ ТЕСТЫ ТОПОЛОГИЧЕСКОЙ СОРТИРОВКИ.....79

ВВЕДЕНИЕ

Актуальность темы. Самостоятельное обучение стало обычной частью учёбы и работы. Студенты осваивают технологии сверх учебной программы, специалисты переходят в новую профессию, школьники готовятся к экзаменам. Учебных материалов стало больше, чем человек успевает изучить: документация, видеокурсы, статьи, онлайн-платформы [13; 40]. Найти материалы перестало быть главной трудностью. Теперь человеку трудно понять, в каком порядке изучать темы, сколько времени это займёт и успеет ли он к нужному сроку.

У того, кто учится самостоятельно, обычно возникают три проблемы.

1. Неясен порядок изучения. Темы зависят друг от друга: виджеты Flutter не понять без основ Dart, а Present Perfect в английском — без знания глагола *to be*. Если пропустить базовую тему, дальнейшее обучение замедляется.
2. Не учитываются уже имеющиеся знания и личное время. Готовые курсы предлагают один план для всех: опытный ученик тратит время на знакомые темы, а занятый человек не успевает за чужим темпом.
3. План разрушается при отставании. Стоит пропустить несколько занятий, и расписание перестаёт соответствовать действительности. Пересчитывать его вручную долго, и часто человек просто бросает обучение.

Существующие решения закрывают эти проблемы по отдельности. Карты технологий показывают зависимости между темами, но не строят расписание [38]. Языковые сервисы мотивируют ежедневными целями, но план в них задаёт сам сервис [16]. Планировщики задач заставляют расписывать всё вручную [32; 41]. Анализ аналогов приведён в п. 1.4. Мобильного приложения, которое автоматически строит персональный план по дням с учётом зависимостей тем, знаний пользователя, его времени и

дедлайна и само перестраивает план при отставании, среди рассмотренных решений не найдено.

Для такой задачи подходит именно мобильное приложение. Телефон всегда под рукой, и пользователь может отметить занятие сразу после него, получить напоминание и увидеть план на сегодня. Это объясняет актуальность темы курсового проекта.

Цель работы — разработать мобильное приложение «Маршрут», которое автоматически формирует персональный план обучения по выбранному направлению с учётом уровня знаний пользователя, доступного времени и срока, а также перестраивает план при отставании.

Задачи работы:

1. Проанализировать предметную область: способы организации самостоятельного обучения, методики распределённого повторения и тайм-менеджмента.
2. Рассмотреть особенности мобильных приложений и принципы их разработки.
3. Проанализировать аналогичные решения и выявить их недостатки.
4. Обосновать выбор технологий: платформы, архитектуры, способа управления состоянием, хранилища данных.
5. Разработать алгоритм формирования плана: граф тем, исключение известных тем по результатам теста, топологическая сортировка, раскладка по дням с учётом лимита часов, проверка дедлайна и интервальные повторения.
6. Спроектировать интерфейс и реализовать экраны приложения: заставку, регистрацию, авторизацию, главную страницу, навигацию, профиль, основной раздел с планом и выход из системы.
7. Протестировать приложение модульными, виджет- и интеграционными тестами и исправить найденные ошибки.
8. Проанализировать первую версию приложения, устранить найденные недостатки и подготовить релизную сборку для Android и Windows.

Объект разработки — процесс планирования самостоятельного обучения.

Предмет разработки — мобильное приложение, которое автоматически формирует и сопровождает персональный план обучения.

Целевая аудитория:

- студенты колледжей и вузов, которые осваивают технологии сверх программы или готовятся к экзаменам;
- самоучки, изучающие программирование или иностранный язык;
- люди, которые меняют профессию и учатся параллельно с работой.

Общее у этих пользователей одно: они учатся без преподавателя, у них ограничено время и, как правило, есть срок, к которому нужно успеть.

Основной функционал приложения:

- регистрация и вход по email и паролю; данные хранятся только на устройстве, пароль сохраняется в виде хэша с солью;
- выбор направления обучения из каталога: Flutter-разработка, Python, английский язык, алгоритмы и структуры данных;
- короткий диагностический тест, по результатам которого известные темы исключаются из плана;
- настройка расписания: часов в неделю, учебных дней и дедлайна;
- автоматическое построение плана по дням с учётом зависимостей тем и интервальными повторениями;
- проверка дедлайна и подсказка, сколько часов в неделю нужно, чтобы успеть;
- отметка выполненных занятий и перестройка плана одной кнопкой при отставании;
- главная страница с прогрессом и задачами на сегодня, статистика, профиль пользователя.

Методы разработки: анализ и сравнение аналогов, объектно-ориентированное проектирование, методы теории графов (топологическая сортировка), модульное и виджет-тестирование.

Практическая значимость работы. Приложение можно использовать для реальной самостоятельной подготовки: уже сейчас в каталоге четыре направления и 75 тем с оценкой трудоёмкости и зависимостями. Приложение работает без интернета и облачных сервисов, не требует ключей и платных подписок и запускается командой `flutter run`. Каталог хранится в JSON-файлах, поэтому новое направление добавляется без изменения кода. Алгоритм планирования выделен в отдельный слой на чистом Dart и покрыт тестами, поэтому его можно повторно использовать в других проектах, например в веб-версии или на сервере.

Структура работы. Курсовая работа состоит из введения, двух глав, заключения, списка использованной литературы и приложений. В первой главе рассмотрены предметная область, особенности и принципы разработки мобильных приложений, проведён анализ аналогов и обоснован выбор технологий. Во второй главе описаны архитектура, алгоритм, интерфейс, реализация и тестирование приложения.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ПРИЛОЖЕНИЯ ДЛЯ ФОРМИРОВАНИЯ ПЕРСОНАЛЬНОГО ПЛАНА ОБУЧЕНИЯ

1.1 Анализ предметной области

1.1.1 Персональный план обучения

Персональный план обучения — это расписание изучения тем, составленное под конкретного человека с учётом его цели, уровня подготовки, доступного времени и срока. В отличие от учебной программы курса, которая одинакова для всех слушателей, персональный план отвечает на три вопроса:

1. **Что изучать?** Набор тем по выбранному направлению без тех, которые человек уже знает.
2. **В каком порядке?** Последовательность, в которой каждая тема идёт после тем, на которые она опирается.
3. **Когда?** Распределение тем по дням в пределах времени, которое человек готов тратить, и с учётом дедлайна.

Чтобы автоматизировать составление плана, предметную область нужно описать формально. Направление обучения разбивается на темы. Для каждой темы известны:

- трудоёмкость, то есть примерное время на изучение в часах;
- уровень сложности;
- категория (например, «Основы», «Интерфейс», «Данные»);
- список тем-предпосылок, без которых тему не понять;
- учебные материалы;
- контрольный вопрос для диагностики (не у всех тем).

Темы вместе с зависимостями образуют **ориентированный граф**: вершины — темы, дуга A → B означает, что тему A нужно изучить раньше темы B. Порядок изучения должен соблюдать все дуги, поэтому в графе не может быть циклов. Если тема A требует B, а B требует A, изучить их

невозможно. Такой граф называется ориентированным ациклическим графом (DAG, Directed Acyclic Graph). Последовательность его вершин, в которой каждая дуга ведёт «вперёд», называется топологической сортировкой [1].

1.1.2 Топологическая сортировка: алгоритм Кана

Один из классических алгоритмов топологической сортировки предложил А. Кан в 1962 году [6]. Алгоритм работает так:

1. Для каждой вершины считается *входящая степень* — число дуг, которые в неё входят. Для темы это число ещё не изученных предпосылок.
2. Вершины с нулевой входящей степенью, то есть темы без предпосылок, помещаются в очередь.
3. Пока очередь не пуста, из неё извлекается вершина и добавляется в результат. У всех вершин, в которые из неё ведут дуги, входящая степень уменьшается на единицу. Вершины, у которых она стала нулевой, добавляются в очередь.
4. Если в результат попали не все вершины, в графе есть цикл, и упорядочить такие темы нельзя.

Каждая вершина и каждая дуга обрабатываются один раз, поэтому время работы алгоритма — O(V + E), где V — число тем, E — число зависимостей [1]. Для каталога из 15–25 тем это доли миллисекунды даже на слабом телефоне.

Алгоритм Кана удобен тем, что сразу обнаруживает циклы. Если в каталоге по ошибке появится взаимная зависимость, приложение не зависнет и не построит неверный план, а сообщит об ошибке данных. Кроме того, если вместо простой очереди использовать очередь с приоритетом, можно управлять порядком среди тем, которые одновременно доступны для изучения. Например, брать сначала более лёгкие темы. Это делает результат детерминированным: для одних и тех же данных план всегда одинаков, что важно для тестирования.

1.1.3 Распределённое повторение

Изученный материал постепенно забывается. Экспериментально это впервые исследовал Г. Эббингауз: он показал, что доля сохранённого материала быстро снижается в первые часы и дни после заучивания, а затем убывает медленнее (так называемая «кривая забывания») [5]. Повторение замедляет забывание. При этом важно не только сколько повторять, но и когда.

Метаанализ Н. Сепеды и соавторов, обобщивший сотни экспериментов, подтвердил *эффект распределённого повторения*: при одинаковом суммарном времени занятия, разнесённые по времени, дают лучшее долговременное запоминание, чем те же занятия подряд [3]. На этом принципе построены системы интервальных повторений: к теме возвращаются через возрастающие промежутки времени.

В приложении «Маршрут» после изучения каждой темы автоматически планируются короткие повторения через 1, 3 и 7 дней. Интервалы выбраны как простое и понятное пользователю приближение к принципу возрастающих промежутков. Подбор оптимальных интервалов для каждого пользователя выходит за рамки курсового проекта.

1.1.4 Тайм-менеджмент: техника Pomodoro

Составить план недостаточно: его нужно выполнять. Для концентрации во время занятия часто используют технику Pomodoro, которую предложил Ф. Чирилло в конце 1980-х годов [4]. Работа делится на интервалы фокуса, классически по 25 минут, между ними делаются короткие перерывы, а после четырёх интервалов — длинный перерыв. Таймер Pomodoro дополняет план: план говорит, *что* изучать сегодня, а таймер помогает сосредоточиться на этом.

1.1.5 Требования к приложению

Из анализа предметной области следуют основные требования к приложению (таблица 1.1).

Таблица 1.1 — Требования к приложению и их источники

№	Требование	Откуда следует
1	Хранить каталог направлений и тем с трудоёмкостью и зависимостями	Формальная модель предметной области (п. 1.1.1)
2	Определять уже известные темы с помощью короткого теста	Проблема «один план для всех»
3	Упорядочивать темы с учётом зависимостей и обнаруживать ошибки в данных	Граф тем, алгоритм Кана (п. 1.1.2)
4	Раскладывать темы по выбранным дням с лимитом времени в день	Учёт личного времени
5	Проверять дедлайн и подсказывать нужную нагрузку	Учёт срока
6	Планировать повторения	Эффект распределённого повторения (п. 1.1.3)
7	Перестраивать план при пропусках, сохраняя выполненное	Проблема «план разрушается при отставании»
8	Отмечать занятия, показывать прогресс, напоминать о занятиях, вести таймер	Выполнение плана, мотивация (п. 1.1.4)
9	Работать без интернета, хранить данные на устройстве, защищать вход паролем	Личные данные, доступность в любое время

1.2 Особенности мобильных приложений

Мобильные приложения разрабатываются с учётом ряда особенностей, которых нет у настольных программ и веб-сайтов.

Ограниченный экран и сенсорный ввод. На экране телефона помещается немного информации, а управление идёт пальцем, а не точным курсором мыши. Поэтому элементы управления должны быть достаточно крупными, на экране должно быть только главное, а второстепенное нужно выносить на отдельные экраны. В рекомендациях Material Design описаны минимальные размеры области касания и типовые компоненты навигации: нижняя панель, вкладки, плавающая кнопка действия [31].

Разные размеры экрана и ориентация. Одно и то же приложение запускается на компактных телефонах, планшетах, складных устройствах, а Flutter-приложение ещё и в окне настольной системы [24]. Интерфейс должен приспосабливаться к ширине окна. В рекомендациях Android окна делятся на

классы по ширине: компактный (до 600 dp), средний (от 600 до 840 dp) и расширенный (от 840 dp) [43]. На компактном экране навигацию принято размещать внизу, на расширенном — сбоку.

Жизненный цикл и прерывания. Пользователь может в любой момент свернуть приложение, получить звонок или переключиться на другую программу. Операционная система вправе завершить процесс свёрнутого приложения, если ей не хватает памяти [34]. Поэтому важные данные нужно сохранять сразу, а не «при выходе», а после повторного запуска восстанавливать состояние, например автоматически входить в аккаунт.

Работа без сети. Мобильный интернет бывает медленным или недоступным: в транспорте, в метро, в поездке. Приложение, которое без сети перестаёт работать, теряет пользователя. Подход offline-first, при котором данные хранятся на устройстве, делает приложение доступным всегда.

Уведомления. Мобильные платформы позволяют приложению напомнить о себе, даже когда оно закрыто. Для приложения, где важна регулярность, это ключевая функция. На Android 13 и новее для показа уведомлений нужно отдельное разрешение пользователя, поэтому его нужно запрашивать в момент, когда пользователь понимает, зачем оно нужно [26].

Ограниченные ресурсы. Аккумулятор, память и процессор телефона ограничены. Тяжёлые вычисления и постоянная работа в фоне быстро разряжают батарею. Алгоритмы приложения должны быть эффективными, а фоновые задачи нужно поручать системе, например планировщику уведомлений.

Безопасность данных. Телефон легко потерять, им могут пользоваться несколько человек. Пароли нельзя хранить в открытом виде. Минимальная мера — хранить хэш пароля с уникальной случайной солью, чтобы одинаковые пароли давали разные хэши и нельзя было воспользоваться заранее вычисленными таблицами [33].

Несколько платформ. Основные мобильные платформы — Android и iOS, у каждой свой язык разработки, набор инструментов и правила оформления. Разработка двух отдельных приложений удваивает затраты, поэтому популярны кроссплатформенные фреймворки, позволяющие писать один код для нескольких платформ (п. 1.5.1).

1.3 Принципы разработки мобильных приложений

С учётом перечисленных особенностей при разработке используются следующие принципы.

1. Разделение ответственности (слоистая архитектура). Код приложения делится на слои, каждый из которых отвечает за своё. В подходе «чистой архитектуры» Р. Мартина бизнес-правила находятся в центре и не зависят от интерфейса, базы данных и фреймворков; зависимости направлены только внутрь [2]. Для мобильного приложения обычно выделяют три слоя:

- *presentation* — экраны и управление их состоянием;
- *domain* — сущности, бизнес-логика и сценарии использования;
- *data* — работа с базой данных, файлами и сетью.

Документация Flutter также рекомендует разделять приложение на слой интерфейса и слой данных, а сложную логику выносить в отдельные классы [21]. Такое разделение позволяет тестировать логику без запуска интерфейса и заменять хранилище, не переписывая экраны.

2. Однонаправленный поток данных и явное управление состоянием. Состояние приложения (текущий пользователь, план, список занятий) хранится в одном месте. Интерфейс строится по состоянию и меняет его только через определённые методы. В Flutter для этого используются библиотеки управления состоянием: Provider, Riverpod, BLoC и другие [23]. Явное состояние делает поведение предсказуемым и упрощает поиск ошибок.

3. Декларативный интерфейс. Во Flutter интерфейс описывается как функция от состояния: при изменении данных фреймворк сам перестраивает нужные виджеты [9]. Разработчику не нужно вручную синхронизировать экран с данными, что снижает количество ошибок.

4. Единая система дизайна. Все экраны оформляются по одним правилам: цвета, шрифты, отступы, формы кнопок. Material Design 3 задаёт цветовые схемы, типографику и компоненты для светлой и тёмной темы [31]. Во Flutter это реализуется через объект `ThemeData`, поэтому стиль задаётся один раз для всего приложения.

5. Адаптивность. Вёрстка строится от доступной ширины, а не от фиксированных размеров: на широком экране навигация переносится вбок, а содержимое ограничивается по ширине, чтобы строки не растягивались [43].

6. Обработка ошибок и пустых состояний. Каждый экран, загружающий данные, должен показывать три состояния: загрузку, ошибку с понятным сообщением и пустое состояние с подсказкой, что делать дальше. Ошибки ввода показываются рядом с полем, а не в общем окне.

7. Защита закрытых разделов. Экраны, доступные только после входа, защищаются на уровне навигации: при попытке открыть их без авторизации пользователь перенаправляется на экран входа. Проверка в одном месте, в маршрутизаторе, надёжнее, чем проверки на каждом экране.

8. Тестирование. Различают три уровня автоматических тестов: модульные (проверяют функцию или класс), виджет-тесты (проверяют экран без устройства) и интеграционные (проверяют приложение целиком на устройстве или эмуляторе) [9]. Чем больше логики вынесено в чистые функции, тем больше её покрывается быстрыми модульными тестами.

1.4 Анализ аналогичных решений

Для анализа выбраны решения, которыми пользуются при самостоятельном обучении: сайт карт изучения технологий roadmap.sh [38],

сервис изучения языков Duolingo [16], образовательные платформы Stepik [40] и Coursera [13], а также планировщики задач Notion [32] и Todoist [41].

roadmap.sh — открытый проект с картами изучения технологий: frontend, backend, Python, Flutter и других. Карта показывает темы и связи между ними. Сильная сторона — наглядный граф тем, составленный сообществом. Недостаток — карта одинакова для всех: она не учитывает, что человек уже знает, сколько у него времени и к какому сроку нужно успеть.

Duolingo — популярный сервис для изучения иностранных языков; позднее в нём появились курсы математики и музыки. Сильные стороны — ежедневная цель, серия дней подряд и напоминания. Порядок уроков задаёт сам сервис; есть входной тест уровня. Расписание под дедлайн пользователя не строится, набор направлений ограничен.

Stepik и Coursera — платформы онлайн-курсов. Внутри курса темы упорядочены автором, у некоторых курсов есть сроки сдачи заданий. Платформы дают сами материалы, но не составляют личный план: темп и порядок определяет курс, а не пользователь.

Notion и Todoist — универсальные инструменты для заметок и задач. В них можно составить любой план, но только вручную, без учёта зависимостей тем и без автоматической перестройки при пропусках.

Результаты сравнения сведены в таблицу 1.2.

Таблица 1.2 — Сравнение аналогов

Возможность	**roadmap.sh**	**Duolingo**	**Stepik / Coursera**	**Notion / Todoist**	**«Маршрут»**
Порядок тем с учётом зависимостей	да, статичная схема	внутри курса	внутри курса	нет	да, автоматически
Учёт уже имеющихся знаний	нет	частично, тест уровня	нет	нет	да, диагностический тест
План по дням под личное время	нет	ежедневная цель	сроки курса	вручную	да
Проверка дедлайна и подсказка нагрузки	нет	нет	нет	нет	да
Перестройка плана при отставании	нет	нет	нет	вручную	да, одной кнопкой
Интервальные повторения	нет	да	нет	нет	да
Работа без интернета и облачной учётной записи	нет	нет	нет	частично	да

Сведения об аналогах составлены по их публичным описаниям на момент написания работы. Сервисы регулярно обновляются: перед защитой сверить таблицу с актуальными версиями. [ПРОВЕРИТЬ]

Вывод по анализу аналогов. Каждое из рассмотренных решений закрывает часть задачи. roadmap.sh даёт граф тем, Duolingo мотивирует регулярностью, платформы курсов дают материалы, планировщики позволяют вести расписание. Ни одно из них не объединяет все шаги: граф тем, учёт знаний, расписание под время и срок пользователя и автоматическую перестройку. Эти возможности и составляют основу разрабатываемого приложения «Маршрут».

1.5 Обоснование выбора технологий

1.5.1 Платформа разработки: Flutter и Dart

Для разработки мобильного приложения рассматривались три подхода:

1. **Нативная разработка** — Kotlin для Android и Swift для iOS. Даёт полный доступ к возможностям платформы, но требует двух отдельных приложений с двумя кодовыми базами.
2. **React Native** (Meta) — кроссплатформенный фреймворк на JavaScript/TypeScript, использующий нативные компоненты платформы [35].
3. **Flutter** (Google) — кроссплатформенный фреймворк на языке Dart с собственным движком отрисовки интерфейса [9; 24].

Также существует Kotlin Multiplatform от JetBrains, позволяющий выносить общую логику в общий код [30]; интерфейс при этом часто пишется отдельно для каждой платформы или с помощью Compose Multiplatform.

Выбран Flutter по следующим причинам:

- **Одна кодовая база для нескольких платформ.** Из одного проекта собираются приложения для Android и iOS, а также веб-версия и настольные приложения для Windows, macOS и Linux [24]. В рамках проекта из одного кода собираются релизный APK для Android, версия для Windows и версия для Linux; сборка и тесты выполняются автоматически (п. 2.8).
- **Собственная отрисовка.** Flutter рисует интерфейс сам, а не через нативные компоненты, поэтому приложение выглядит одинаково на всех устройствах [7; 9].
- **Быстрая разработка.** Горячая перезагрузка (hot reload) применяет изменения кода за секунды без перезапуска приложения [7].
- **Производительность.** В режиме разработки код Dart выполняется JIT-компилятором, а в релизной сборке компилируется заранее (AOT) в машинный код [14].
- **Богатая библиотека виджетов Material 3** и большое число пакетов на pub.dev.

- Технология изучалась в рамках дисциплины ПМ07, что позволило сосредоточиться на алгоритме и качестве приложения.

Язык Dart — объектно-ориентированный язык со строгой статической типизацией. С версии Dart 3 обязательна *звуковая null-безопасность*: компилятор гарантирует, что переменная без знака `?` не может быть `null` [39]. Это исключает целый класс ошибок времени выполнения. Dart поддерживает асинхронное программирование через `Future`, `async/await` и `Stream`, что используется для работы с базой данных и реактивного обновления экранов [14].

1.5.2 Архитектура: Clean Architecture

Приложение построено по принципам чистой архитектуры с тремя слоями [2; 21]:

- **domain** — сущности (тема, план, занятие), алгоритм формирования плана, интерфейсы репозиториев. Слой написан на чистом Dart и не зависит от Flutter, поэтому алгоритм тестируется быстрыми модульными тестами;
- **data** — реализации репозиториев: база данных SQLite, JSON-каталог тем, настройки, уведомления;
- **presentation** — экраны, виджеты и провайдеры состояния.

Альтернатива — писать логику прямо в виджетах. Для небольших учебных примеров это допустимо, но в приложении с нетривиальным алгоритмом такой код трудно тестировать и развивать. Например, алгоритм пришлось бы проверять через нажатия на кнопки.

1.5.3 Управление состоянием: Riverpod

Рассматривались три распространённых варианта [23].

Таблица 1.3 — Сравнение способов управления состоянием

Критерий	**Provider**	**BLoC**	**Riverpod**
Объём шаблонного кода	малый	большой: события, состояния, блоки	малый
Зависимость от `BuildContext`	да	да (через `BlocProvider`)	нет
Обнаружение ошибок	часть ошибок только во время выполнения	на этапе компиляции	на этапе компиляции
Асинхронные данные	вручную	через состояния блока	встроенный тип `AsyncValue` (загрузка, данные, ошибка)
Подмена зависимостей в тестах	ограниченно	да	да, через `overrides`

Provider прост, но провайдеры ищутся по дереву виджетов, и ошибка «провайдер не найден» проявляется только при запуске. **BLoC** — архитектурный паттерн, который Google представил на конференции DartConf в 2018 году, и реализующая его библиотека flutter_bloc [12]. Он хорошо подходит большим командам, но требует много кода: для каждого действия нужно описать событие и состояние.

Выбран **Riverpod** [37] — библиотека того же автора, что и Provider, созданная для устранения его ограничений:

- провайдеры объявляются как глобальные константы и не зависят от дерева виджетов, поэтому ошибки обнаруживаются при компиляции;
- `AsyncValue` единообразно описывает загрузку, данные и ошибку, что упрощает показ трёх состояний экрана (п. 1.3);
- любой провайдер подменяется в тестах через `ProviderScope(overrides: …)`, например база данных в памяти вместо файла и фиксированная дата вместо текущей;
- провайдеры-потоки (`StreamProvider`) подписываются на изменения в базе данных, и экраны обновляются автоматически.

1.5.4 Хранение данных: drift (SQLite)

Требования к хранилищу: работа без сети, связанные данные (пользователь → планы → темы плана → занятия), выборки с фильтрами и сортировкой, целостность данных при удалении.

Таблица 1.4 — Сравнение хранилищ данных

Критерий	**shared_ preferences**	**Hive**	**Firebase**	**drift (SQLite)**
Модель данных	ключ–значение	ключ–значение / объекты	документы в облаке	реляционные таблицы
Связи и каскадное удаление	нет	вручную	вручную	внешние ключи, `ON DELETE CASCADE`
Запросы с фильтрами и сортировкой	нет	в памяти	да, с ограничениями	SQL
Работа без интернета	да	да	кэш, синхронизация при подключении	да
Нужны ключи и учётная запись	нет	нет	да, проект Firebase	нет
Типобезопасность запросов	нет	частично	нет	да, генерация кода

shared_preferences подходит только для простых настроек; в проекте он хранит идентификатор текущего пользователя и выбранную тему оформления. **Hive** — быстрое NoSQL-хранилище на чистом Dart [29], но связи между объектами и каскадное удаление пришлось бы реализовывать вручную. **Firebase** — облачная платформа Google [17]. Он требует подключения к сети для синхронизации, создания проекта и конфигурационных ключей, что противоречит требованию запускать проект командой `flutter run` без настройки.

Выбран **drift** [15] — библиотека для работы с SQLite. SQLite — встраиваемая реляционная СУБД, которая хранит базу в одном файле на устройстве и не требует сервера [10]. drift генерирует типобезопасный код по описанию таблиц, поддерживает транзакции и реактивные запросы: при

изменении таблицы подписанные экраны получают новые данные автоматически.

1.5.5 Навигация: go_router

go_router — официальный пакет команды Flutter для декларативной навигации [28]. Маршруты описываются как адреса (`/home`, `/plan`, `/session/:id`), что упрощает переходы и передачу параметров. Важная возможность — функция `redirect`: перед каждым переходом проверяется состояние входа, и неавторизованный пользователь перенаправляется на экран входа. Так реализуется требование «для доступа к закрытым разделам требуется авторизация» в одном месте приложения. `StatefulShellRoute` сохраняет состояние каждой вкладки нижней навигации при переключении.

1.5.6 Локальная авторизация: хэш пароля с солью

Учётные записи хранятся только на устройстве. Пароль не сохраняется в открытом виде: при регистрации генерируется случайная соль длиной 16 байт, вычисляется хэш SHA-256 [8] от соли и пароля, в базу записываются соль и хэш. При входе хэш вычисляется заново и сравнивается с сохранённым. Соль делает хэши одинаковых паролей разными.

Так была устроена первая версия приложения (1 семестр). Однако SHA-256 вычисляется очень быстро, и если файл базы скопируют с устройства, простые пароли можно подобрать перебором. Для хранения паролей OWASP рекомендует медленные функции: Argon2id, scrypt, bcrypt, PBKDF2 [33]. Во второй версии приложения выбран PBKDF2-HMAC-SHA256 [36]: функция HMAC-SHA256 применяется к паролю и соли 100 000 раз. PBKDF2 реализуется на основе того же пакета crypto, поэтому дополнительные зависимости не нужны. Хэши первой версии пересчитываются автоматически при входе пользователя (п. 2.6.1).

1.5.7 Дополнительные пакеты

- **fl_chart** [18] — построение столбчатых и круговых диаграмм для экрана статистики.
- **flutter_local_notifications** [26] — локальные уведомления: ежедневное напоминание о занятиях и сигнал окончания интервала Pomodoro. Уведомления формируются на устройстве и не требуют сервера.
- **Material 3** [31] — система дизайна с поддержкой светлой и тёмной темы. Для приложения выбрана фирменная цветовая схема: тёмно-синий основной цвет и оранжевый акцентный.

Таблица 1.5 — Итоговый стек технологий

Задача	**Технология**
Язык и фреймворк	Dart, Flutter
Архитектура	Clean Architecture: domain / data / presentation
Управление состоянием	Riverpod
База данных	drift (SQLite)
Простые настройки	shared_preferences
Навигация	go_router с redirect
Хэширование паролей	PBKDF2-HMAC-SHA256 с солью (пакет crypto)
Графики	fl_chart
Уведомления	flutter_local_notifications
Дизайн	Material 3, светлая и тёмная темы
Тестирование	flutter_test, integration_test

Выводы по главе 1

1. Задача составления персонального плана формализована как работа с ориентированным ациклическим графом тем. Порядок изучения находится топологической сортировкой алгоритмом Кана за время O(V + E). Алгоритм позволяет также обнаружить ошибки в данных (циклы).
2. Эффект распределённого повторения обосновывает автоматическое планирование повторений, а техника Pomodoro — встроенный таймер концентрации.
3. Рассмотрены особенности мобильных приложений: ограниченный экран, разные размеры и ориентации, прерывания, работа без сети, уведомления, ограниченные ресурсы, безопасность данных.

Сформулированы принципы разработки, которые учитывают эти особенности.

4. Анализ аналогов показал, что существующие решения закрывают задачу частично. Ни одно не объединяет граф тем, учёт знаний, расписание под время и срок пользователя и автоматическую перестройку плана.
5. Обоснован выбор технологий: Flutter и Dart, Clean Architecture, Riverpod, drift (SQLite), go_router, хэширование паролей PBKDF2 с солью, fl_chart, flutter_local_notifications, Material 3. Стек позволяет создать приложение, которое работает без интернета и облачных сервисов и хорошо тестируется.

ГЛАВА 2. РАЗРАБОТКА ПРИЛОЖЕНИЯ «МАРШРУТ»

2.1 Требования к приложению

На основе анализа предметной области и аналогов (глава 1) сформулированы требования к приложению.

Функциональные требования приведены в таблице 2.1.

Таблица 2.1 — Функциональные требования

№	Требование	Где реализовано
Ф1	Регистрация по имени, email и паролю с проверкой полей; вход; автовход при «Запомнить меня»; выход	Экраны входа и регистрации, `AuthController`
Ф2	Закрытые разделы доступны только после входа	Правило `authRedirect` в `app_router.dart`
Ф3	Выбор направления обучения из каталога	Шаг 1 онбординга
Ф4	Диагностический тест: известные темы исключаются из плана	Шаг 2 онбординга, `KnowledgeEstimator`
Ф5	Настройка расписания: часы в неделю, учебные дни, дедлайн	Шаг 3 онбординга, экран настроек плана
Ф6	Автоматическое построение плана по дням с учётом зависимостей тем, лимита часов и интервальных повторений	`PlanGenerator`, `Scheduler`
Ф7	Проверка дедлайна и подсказка о нужном числе часов	`PlanGenerator`, метод `requiredHoursPerWeek`
Ф8	Отметка занятий, перестройка плана при отставании	Главная, «План», `Replanner`
Ф9	Несколько планов, выбор активного, переименование, удаление	«Мои планы»
Ф10	Свои темы, избранное, поиск и фильтр по категориям	Вкладка «Темы»
Ф11	Заметки к темам с поиском	«Заметки»
Ф12	Статистика: часы по неделям, распределение по категориям, история	Вкладка «Статистика»
Ф13	Таймер Pomodoro с учётом выполненных интервалов	Экран Pomodoro
Ф14	Ежедневное напоминание о занятиях	Профиль, `ReminderService`
Ф15	Профиль: редактирование данных, смена пароля, тема оформления, удаление аккаунта	Раздел «Профиль»

Нефункциональные требования:

- работа без интернета и без серверной части: все данные хранятся на устройстве;
- хранение пароля только в виде хэша с солью;
- платформы: Android (основная), Windows и Linux (настольная версия);

- адаптивный интерфейс: телефон в обеих ориентациях и окно компьютера;
- интерфейс на русском языке в стиле Material 3, фирменные цвета — тёмно-синий и оранжевый;
- покрытие алгоритма и данных автоматическими тестами.

2.2 Архитектура приложения

Приложение построено по принципам чистой архитектуры [2] и рекомендациям команды Flutter по архитектуре приложений [21]. Код разделён на три слоя (рисунок 2.1).

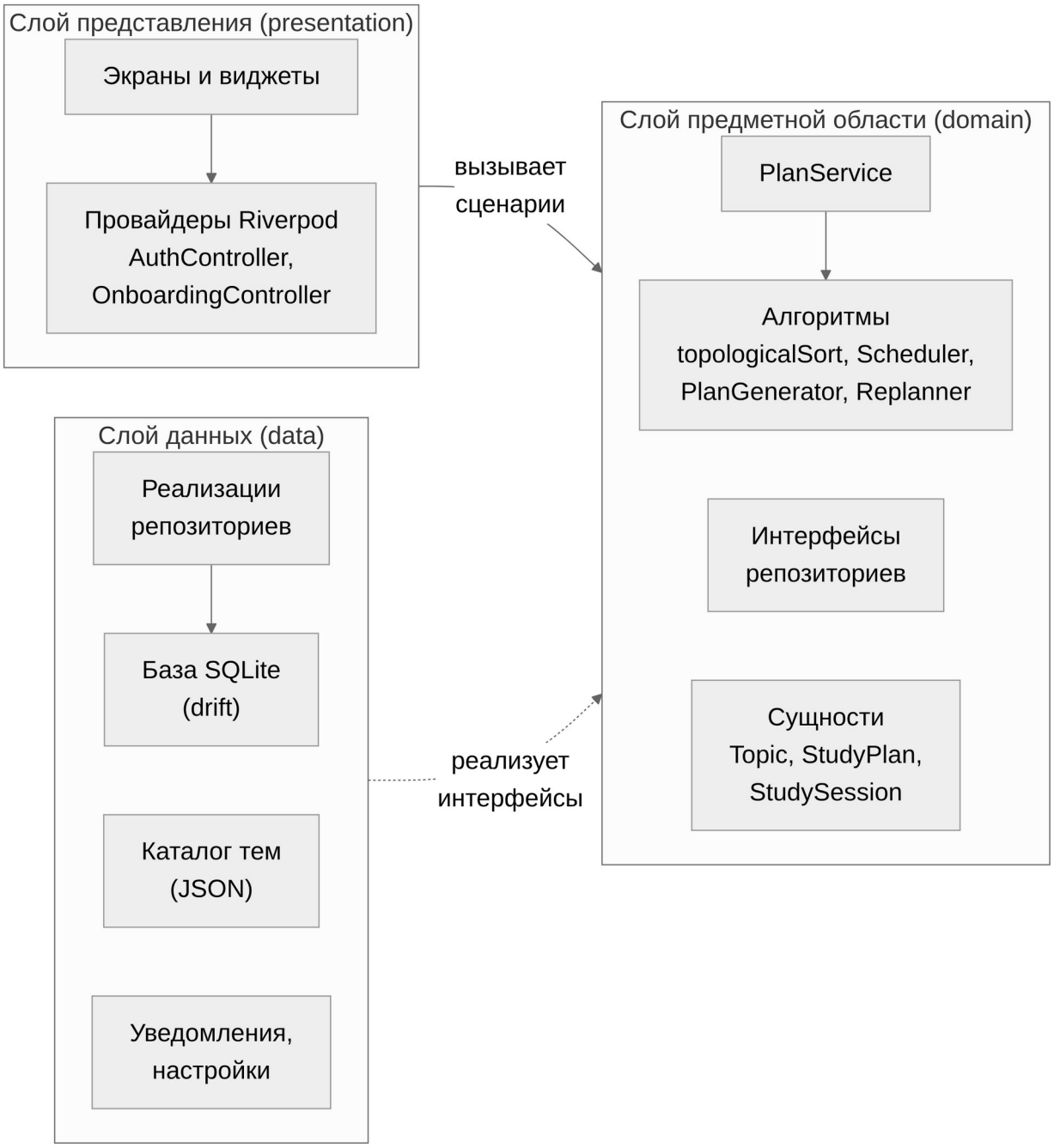


Рисунок 2.1 — Слои приложения и направление зависимостей

1. **Слой предметной области** (`lib/domain`) написан на чистом Dart и не зависит от Flutter и базы данных. В нём находятся сущности (тема, план, занятие, настройки расписания), алгоритм построения плана и интерфейсы репозиториев, то есть описание того, какие данные нужны алгоритмам, без указания, где они хранятся. Поэтому алгоритм проверяется обычными модульными тестами за доли секунды.

2. **Слой данных** (`lib/data`) реализует интерфейсы репозиториев: таблицы drift, чтение каталога из JSON, хэширование паролей, уведомления, хранение сессии в `shared_preferences`.
3. **Слой представления** (`lib/presentation`) содержит экраны и провайдеры Riverpod, которые связывают экраны с данными.

Зависимости направлены внутрь: экраны знают о предметной области, слой данных реализует её интерфейсы, а предметная область ни от кого не зависит. Благодаря этому можно, например, заменить SQLite на облачную базу, не меняя алгоритм и экраны.

Структура проекта приведена в таблице 2.2.

Таблица 2.2 — Структура проекта

Папка	**Назначение**
`lib/core/`	Тема оформления (`theme`), маршрутизация (`router`), валидаторы, форматирование дат и чисел, хэширование паролей (`utils`)
`lib/domain/entities/`	Сущности: `Topic`, `Direction`, `StudyPlan`, `StudySession`, `ScheduleSettings`, `AppUser`, `Note`
`lib/domain/planner/`	Алгоритм: топологическая сортировка, раскладка по дням, генератор плана, перепланирование, оценка знаний по тесту
`lib/domain/repositories/`	Интерфейсы репозиториев и сервисов
`lib/domain/usecases/`	`PlanService`: сценарии создания плана, отметки занятий и перепланирования
`lib/data/db/`	Описание таблиц drift и сгенерированный код запросов
`lib/data/repositories/`	Реализации репозиториев: пользователи, планы, темы, заметки, сессия
`lib/data/datasources/`	Чтение каталога направлений из JSON
`lib/data/services/`	Уведомления (flutter_local_notifications)
`lib/presentation/providers/`	Провайдеры Riverpod и контроллеры состояния
`lib/presentation/screens/`	Экраны, сгруппированные по разделам
`lib/presentation/widgets/`	Общие виджеты: логотип, кольцо прогресса, пустые состояния, карточка занятия
`assets/catalog/`	Каталог: 4 направления, 75 тем, 32 вопроса диагностического теста
`test/`, `integration_test/`	Автоматические тесты
`tool/`	Генераторы скриншотов и иконок, пример работы алгоритма

Объём исходного кода приложения без сгенерированных файлов — около 9 000 строк на Dart, тестов и вспомогательных программ — около 2 650 строк.

Управление состоянием и внедрение зависимостей. Все зависимости объявлены провайдерами Riverpod [37] в файле `lib/presentation/providers/providers.dart`. Внешние ресурсы (база данных, настройки, уведомления, часы) объявлены как провайдеры, которые переопределяются при запуске: в `main()` — настоящими объектами, в тестах — базой в памяти, заглушкой уведомлений и фиксированной датой. Данные из базы передаются в интерфейс потоками (`StreamProvider`): когда пользователь отмечает занятие, drift сообщает об изменении таблицы, и главная, план и статистика обновляются сами, без ручного обновления экранов.

2.3 Проектирование базы данных

Данные хранятся в локальной базе SQLite, работа с ней ведётся через библиотеку drift [15]. Структура базы показана на рисунке 2.2: на диаграмме оставлены ключи и основные поля, чтобы её можно было прочитать на странице. Назначение таблиц описано в таблице 2.3, полное описание всех полей — в приложении Е.

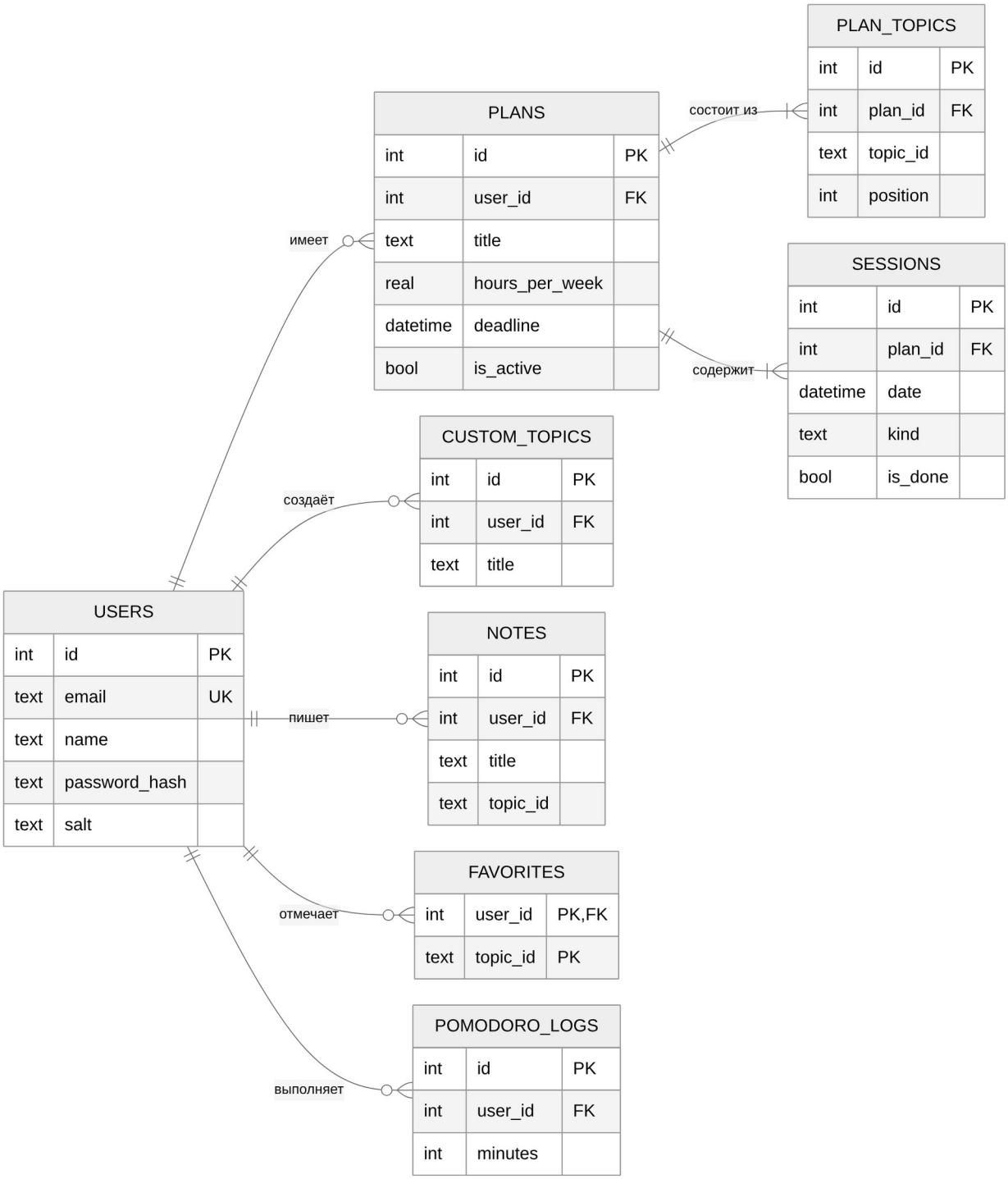


Рисунок 2.2 — ER-диаграмма базы данных (ключи и основные поля)

Таблица 2.3 — Таблицы базы данных

Таблица	**Назначение**
`users`	Учётные записи. Email уникален; пароль хранится как хэш PBKDF2 и соль; настройки напоминания
`plans`	Планы обучения. У пользователя может быть несколько планов, активен один (`is_active`). Хранятся настройки расписания и темы, исключённые по тесту
`plan_topics`	Темы плана в порядке изучения (`position`). Это снимок темы на момент создания плана, поэтому изменение каталога в новой версии приложения не ломает уже созданные планы
`sessions`	Занятия: дата, длительность, вид (`study` — изучение, `review` — повторение), номер части темы, отметка о выполнении и её время
`custom_topics`	Собственные темы пользователя с длительностью, категорией и зависимостями
`notes`	Заметки; могут быть привязаны к теме
`favorites`	Избранные темы; составной первичный ключ (пользователь, тема) исключает повторы
`pomodoro_logs`	Завершённые интервалы Pomodoro для статистики

Все внешние ключи объявлены с правилом `ON DELETE CASCADE`, а при открытии базы включается проверка внешних ключей (`PRAGMA foreign_keys = ON`). Поэтому удаление плана удаляет его темы и занятия, а удаление аккаунта — все данные пользователя одной командой. Это проверено тестами (раздел 2.7).

Каталог направлений не хранится в базе: это JSON-файлы в папке `assets/catalog/`. Каждая тема описана идентификатором, названием, описанием, трудоёмкостью в часах, уровнем сложности (1–6), категорией, списком зависимостей, ссылками на материалы и, для части тем, вопросом диагностического теста. Такой формат позволяет добавить новое направление без изменения кода. В каталоге 4 направления: «Flutter-разработчик» (20 тем), «Python с нуля» (18), «Английский A1–B1» (19) и «Алгоритмы и структуры данных» (18); всего 75 тем, 113 зависимостей и 32 вопроса.

Идентификатор вошедшего пользователя (если включено «Запомнить меня») и выбранная тема оформления хранятся в `shared_preferences`.

2.4 Алгоритм формирования плана

2.4.1 Общая схема

Алгоритм реализован в слое предметной области (`lib/domain/planner/`) и состоит из пяти шагов (рисунок 2.3).

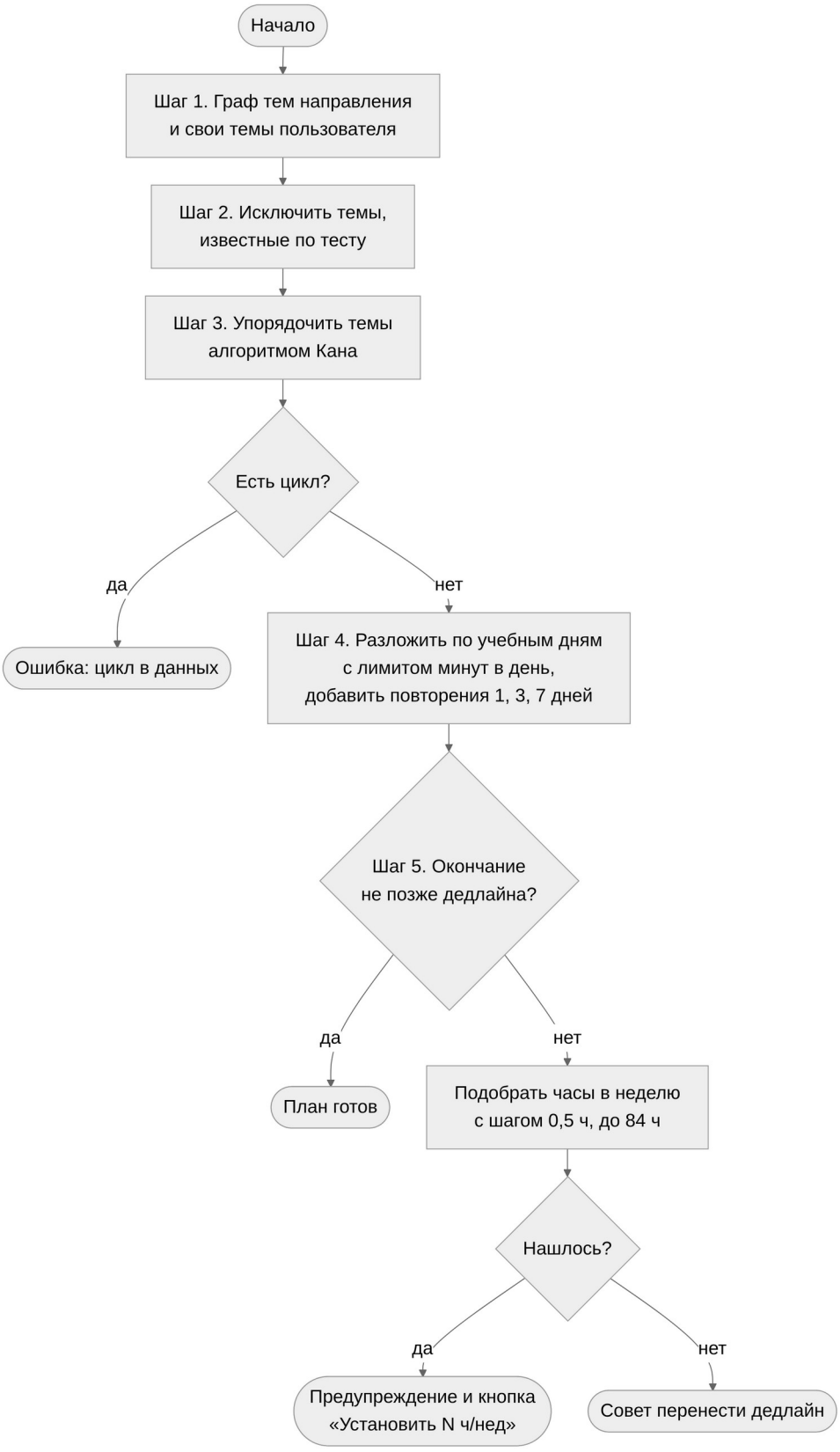


Рисунок 2.3 — Схема алгоритма формирования плана

2.4.2 Исключение известных тем

По ответам диагностического теста класс `KnowledgeEstimator` определяет известные темы. Верный ответ по теме означает, что пользователь знает и саму тему, и все её предшествующие темы: кто правильно ответил о Riverpod, знает и основы Dart. Предшественники находятся обходом графа в глубину. Неверный ответ или «Не знаю» исключает тему из известных, даже если её «покрыл» верный ответ по более сложной теме: явный провал важнее косвенного вывода.

2.4.3 Топологическая сортировка

Темы образуют ориентированный граф: ребро «A → B» означает, что тему B нельзя изучать раньше темы A. Порядок изучения — это топологическая сортировка графа, которая выполняется алгоритмом Кана [1; 6]:

1. Для каждой темы считается входящая степень — число её зависимостей внутри плана. Зависимость, которой нет в плане (тема уже известна), считается выполненной.
2. Темы с нулевой степенью помещаются в очередь.
3. Из очереди извлекается тема и добавляется в результат; у всех зависящих от неё тем степень уменьшается на единицу, и темы с нулевой степенью попадают в очередь.
4. Если в результат попали не все темы, в графе есть цикл, и алгоритм сообщает, какие темы в него входят.

Чтобы результат был одинаковым при одинаковых данных, очередь упорядочена по уровню сложности темы, а при равном уровне — по позиции темы в каталоге. Для этого используется упорядоченное множество `SplayTreeSet`, поэтому сложность алгоритма — $O((V + E) \cdot \log V)$, где V — число тем, E — число зависимостей. Для каталога из 20 тем это мгновенно. Фрагмент реализации:

```
final queue = SplayTreeSet<int>(compare);
for (var i = 0; i < topics.length; i++) {
```

```
  if (inDegree[i] == 0) queue.add(i);
}
final result = <Topic>[];
while (queue.isNotEmpty) {
  final current = queue.first;
  queue.remove(current);
  result.add(topics[current]);
  for (final next in dependents[current]) {
    inDegree[next]--;
    if (inDegree[next] == 0) queue.add(next);
  }
}
if (result.length != topics.length) throw CycleDetectedException(...);
```

Полный текст функции приведён в приложении А.

2.4.4 Раскладка по дням и интервальные повторения

Класс `Scheduler` моделирует календарь день за днём, начиная с даты старта:

1. Дни недели, не выбранные пользователем, пропускаются.
2. Дневной лимит равен часам в неделю, делённым на число учебных дней. Он округляется до 15 минут и не бывает меньше 30 минут.
3. В начале дня ставятся повторения, срок которых наступил, по 15 минут каждое.
4. Оставшееся время заполняется текущей темой. Тема может делиться на части, но часть не бывает короче 30 минут, если тема на ней не заканчивается: короткие обрывки неудобны для занятия.
5. Когда тема изучена полностью, планируются её повторения через 1, 3 и 7 календарных дней. Интервалы выбраны по кривой забывания Эббингауза [5] и исследованиям распределённого повторения [3]. Повторение, выпавшее на неучебный день, переносится на ближайший учебный.

2.4.5 Проверка дедлайна

Датой окончания плана считается день последнего занятия по новой теме (повторения после него не учитываются). Если дата окончания позже дедлайна, генератор подбирает минимальное число часов в неделю, при котором план укладывается в срок: значения перебираются с шагом 0,5 часа

до 84 часов в неделю (12 часов × 7 дней). Если не хватает даже 84 часов, пользователю предлагается перенести дедлайн.

2.4.6 Пример работы алгоритма

Пусть в плане три темы по 60 минут: «Основы Dart», «Виджеты» (зависит от «Основ Dart») и «Состояние» (зависит от «Виджетов»). Пользователь выбрал 7 часов в неделю, все дни недели, старт 12.10.2026 и дедлайн 14.10.2026. Дневной лимит: 7 · 60 / 7 = 60 минут. Результат работы алгоритма приведён в таблице 2.4 (получен программой `tool/print_schedule_example.dart`).

Таблица 2.4 — Пример расписания

Дата	**Занятие**	**Минут**
12.10	Основы Dart	60
13.10	Повторение: Основы Dart	15
13.10	Виджеты, часть 1 из 2	45
14.10	Виджеты, часть 2 из 2	15
14.10	Состояние, часть 1 из 2	45
15.10	Повторение: Основы Dart	15
15.10	Повторение: Виджеты	15
15.10	Состояние, часть 2 из 2	15
16.10	Повторение: Состояние	15
17.10	Повторение: Виджеты	15
18.10	Повторение: Состояние	15
19.10	Повторение: Основы Dart	15
21.10	Повторение: Виджеты	15
22.10	Повторение: Состояние	15

Повторения занимают часть дневного лимита, поэтому изучение заканчивается 15.10, на день позже дедлайна. Алгоритм сообщает, что план не укладывается в срок, и предлагает 8 часов в неделю: тогда лимит становится 75 минут, и последняя тема изучается 14.10.

2.4.7 Перепланирование

Если пользователь пропустил занятия или изменил расписание, класс `Replanner` перестраивает план. Выполненные занятия остаются на своих местах. Для каждой темы считается, сколько минут уже изучено, и остаток заново раскладывается начиная с сегодняшнего дня по тем же правилам.

Невыполненные повторения уже изученных тем переносятся на ближайшие учебные дни. Номера частей продолжают уже выполненные части. После перестройки снова проверяется дедлайн.

2.5 Интерфейс приложения

2.5.1 Навигация

Карта экранов показана на рисунке 2.4. Основные разделы доступны через пять вкладок: «Главная», «План», «Темы», «Статистика», «Профиль». Вкладки построены на `StatefulShellRoute` из go_router [28], поэтому каждая вкладка сохраняет своё состояние (прокрутку, фильтры) при переключении.

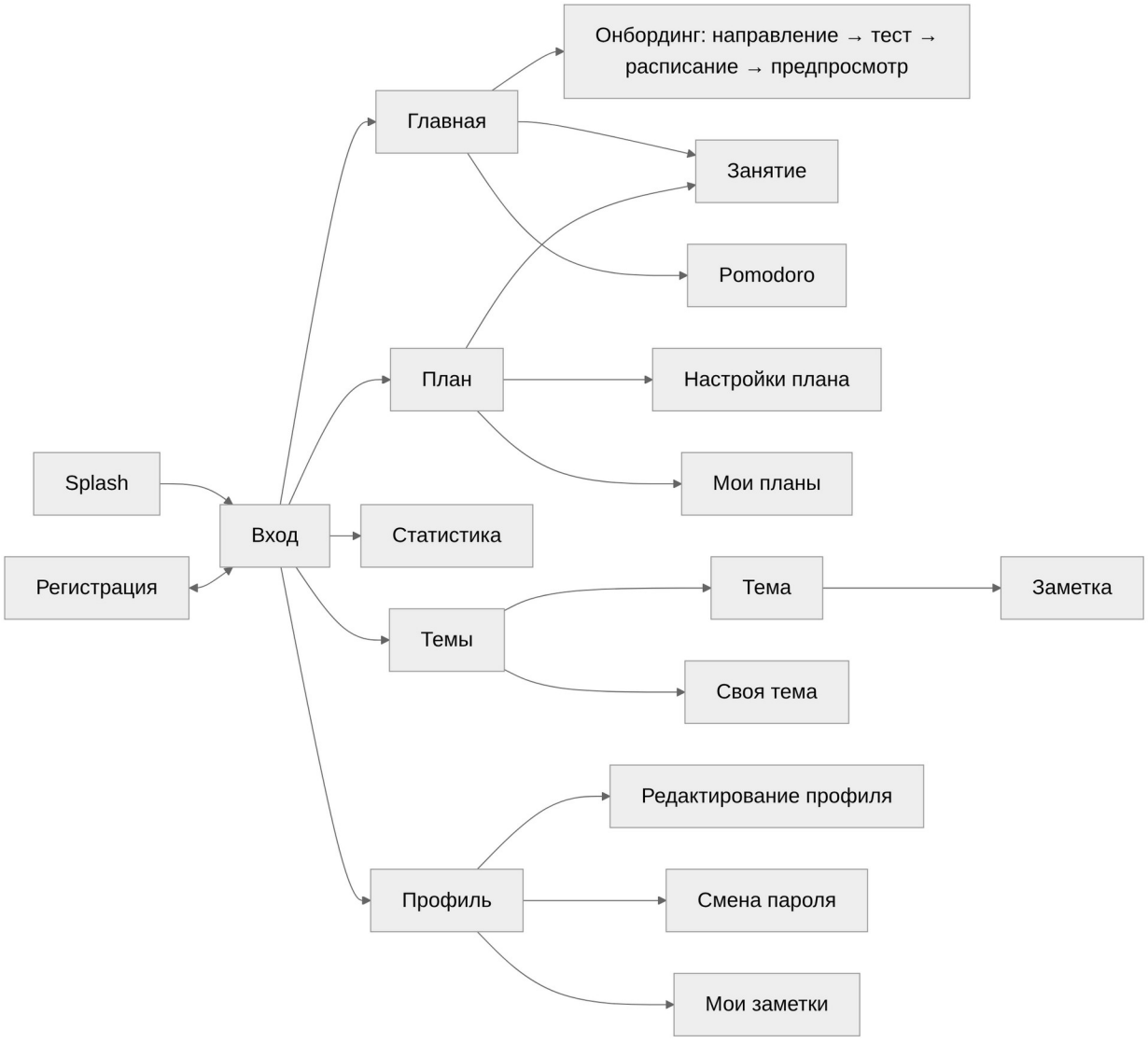


Рисунок 2.4 — Карта экранов

Доступ к экранам проверяет одна функция `authRedirect`, которую go_router вызывает перед каждым переходом:

```
String? authRedirect(AuthStatus status, String location) {
  if (status == AuthStatus.unknown) {
    return location == '/splash' ? null : '/splash';
  }
  final loggedIn = status == AuthStatus.authenticated;
  final isPublic = publicRoutes.contains(location);
  if (!loggedIn) return isPublic ? null : '/login';
  if (isPublic || location == '/splash') return '/home';
  return null;
}
```

Пока сессия проверяется, виден только Splash; без входа доступны только вход и регистрация; после входа экраны входа ведут на главную. Роутер подписан на состояние авторизации, поэтому выход из аккаунта или его удаление сразу открывают экран входа.

2.5.2 Оформление

Интерфейс построен на Material 3 [31]. Цветовая схема создаётся из фирменного тёмно-синего цвета (#1A3A8F), акцентный цвет — оранжевый (#F57C00). Есть светлая и тёмная темы; пользователь выбирает светлую, тёмную или системную тему в профиле. Стили текста строятся от типографики темы, поэтому шрифт и размеры одинаковы на всех экранах. Логотип «М» в виде линии маршрута нарисован кодом (`AppLogo`), иконки приложения для всех платформ генерируются из того же рисунка (`tool/icons/generate_icons.py`).

Пустые состояния (нет плана, нет заметок, ничего не найдено) показывают пояснение и кнопку следующего действия. Ошибки ввода выводятся под полями форм, ошибки операций — во всплывающих сообщениях. Необратимые действия (удаление плана, темы, заметки, выход, удаление аккаунта) требуют подтверждения.

2.5.3 Адаптивность

На широком экране (от 840 логических пикселей, класс «расширенное окно» по рекомендациям Android [43]) нижняя панель вкладок заменяется

боковой (`NavigationRail`), а содержимое экранов ограничивается колонкой шириной 720 пикселей по центру, чтобы строки текста не растягивались на весь монитор. Работа на телефоне в обеих ориентациях и в окне 1280×800 проверяется автоматическим тестом (раздел 2.7). Подписи вкладок не увеличиваются вместе с системным шрифтом, чтобы «Статистика» не переносилась по буквам, а остальной текст масштабируется. Перед сдачей все экраны дополнительно сняты в семи вариантах: светлая и тёмная тема, маленький телефон 360×640, крупный шрифт 130 %, горизонтальная ориентация и окно компьютера; по этим снимкам исправлено десять недочётов (ошибки № 13–22 в разделе о тестировании).

2.5.4 Экраны приложения

Экраны приложения перечислены в таблице 2.5 и показаны на рисунках 2.5–2.29. Скриншоты получены программой `tool/screenshots/screenshots_test.dart` на тестовых данных: пользователь Алия, план «Flutter-разработчик», начатый за четыре недели до текущей даты.

Таблица 2.5 — Экраны приложения

Экран	**Назначение**	**Рисунок**
Splash	Логотип и название, проверка сохранённой сессии	2.5
Вход	Email, пароль, «Запомнить меня», сообщение о неверных данных	2.6, 2.8
Регистрация	Имя, email, пароль и повтор с проверкой каждого поля	2.7
Главная	Приветствие, кольцо прогресса, часы, серия дней, дни до дедлайна, задачи на сегодня, баннер о пропущенных занятиях	2.9
План	Занятия по дням с фильтрами и поиском	2.10
Темы	Каталог направления, поиск, категории, избранное, свои темы	2.11
Тема	Описание, материалы, заметки, избранное	2.12
Статистика	Итоги, график часов по неделям, распределение по категориям, история	2.13
Профиль	Данные пользователя, показатели, настройки, выход	2.14
Мои планы	Список планов, выбор активного, переименование, перестройка, удаление	2.15

Экран	Назначение	Рисунок
Pomodoro	Таймер 25/5 или 50/10 минут, число интервалов за день	2.16
Заметки	Список заметок с поиском	2.17
Занятие	Описание темы, материалы, отметка выполнения, запуск Pomodoro	2.18
Настройки плана	Изменение расписания с перестройкой плана	2.19
Онбординг	Выбор направления, диагностический тест, расписание, предпросмотр	2.20–2.23
Тёмная тема	Главная и статистика в тёмной теме	2.24, 2.25
Окно Windows	Главная и план с боковой панелью вкладок	2.26, 2.27
Горизонтальная ориентация	Статистика на телефоне в горизонтальной ориентации	2.28
Удаление аккаунта	Подтверждение паролем	2.29

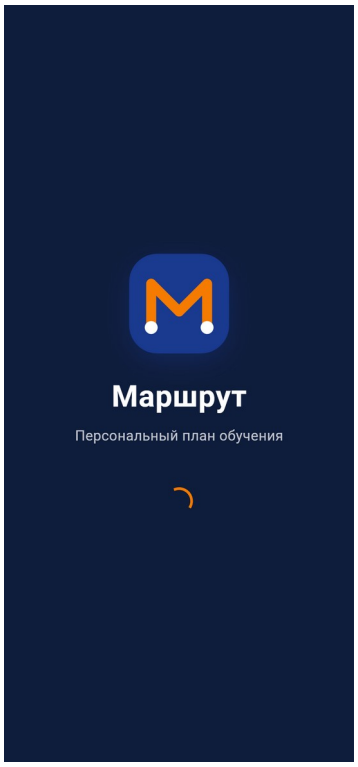


Рисунок 2.5 — Заставка

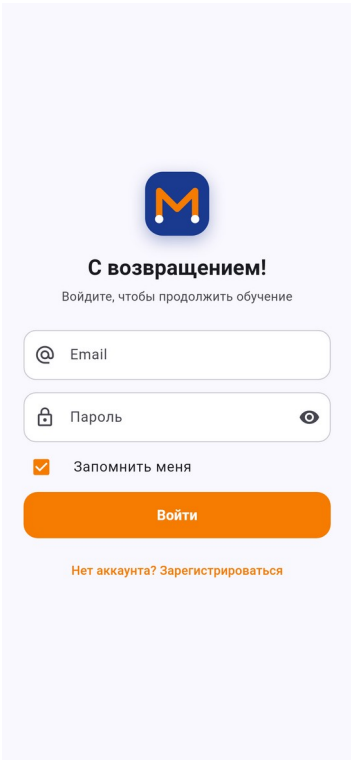


Рисунок 2.6 — Экран входа

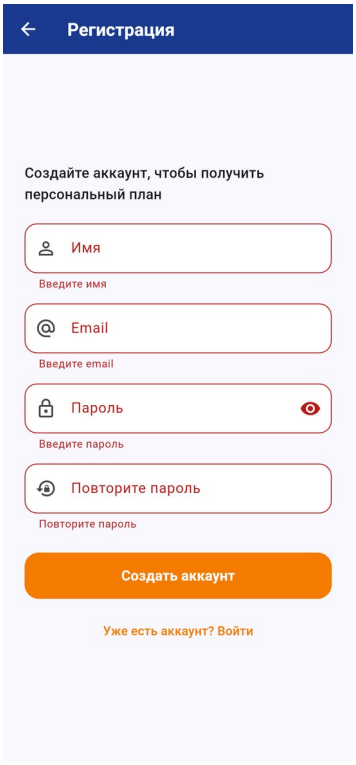


Рисунок 2.7 — Регистрация: сообщения об ошибках под полями

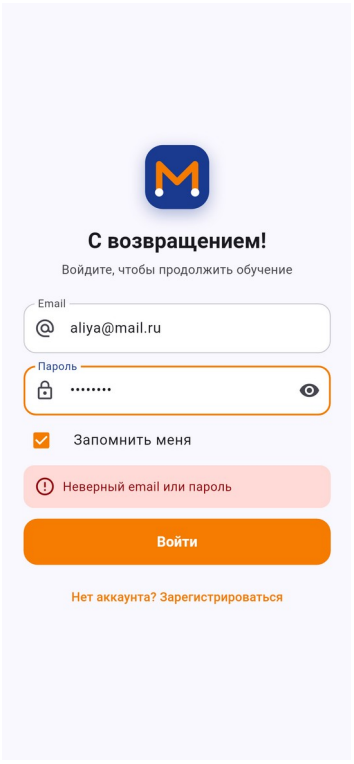


Рисунок 2.8 — Вход с неверным паролем

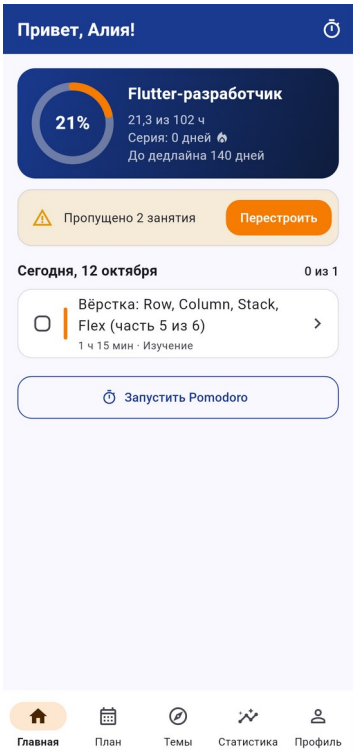


Рисунок 2.9 — Главная страница

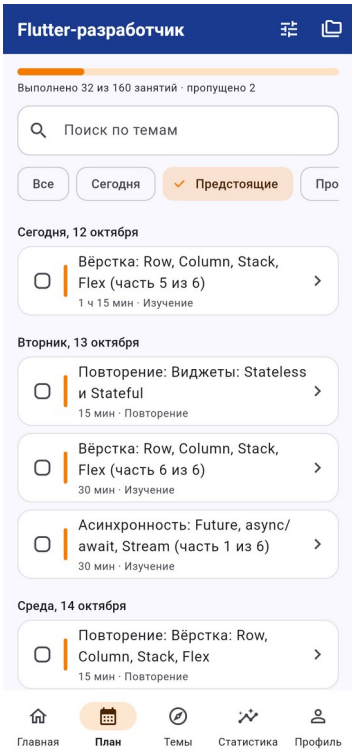


Рисунок 2.10 — План по дням

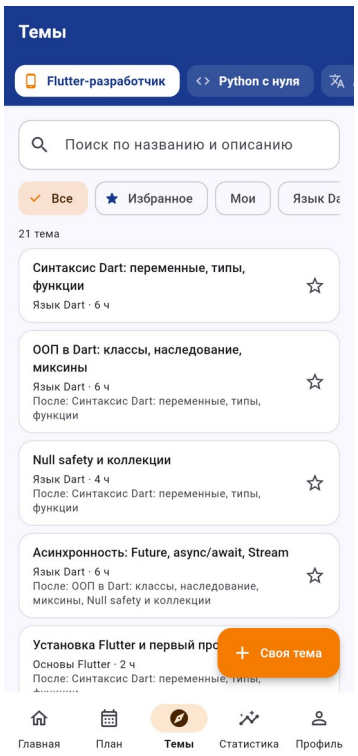


Рисунок 2.11 — Вкладка «Темы»

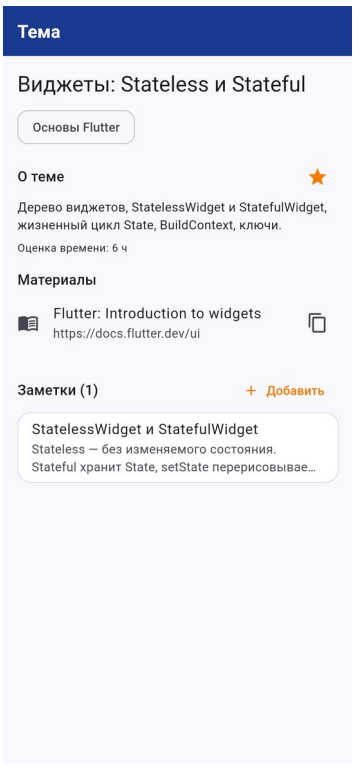


Рисунок 2.12 — Экран темы

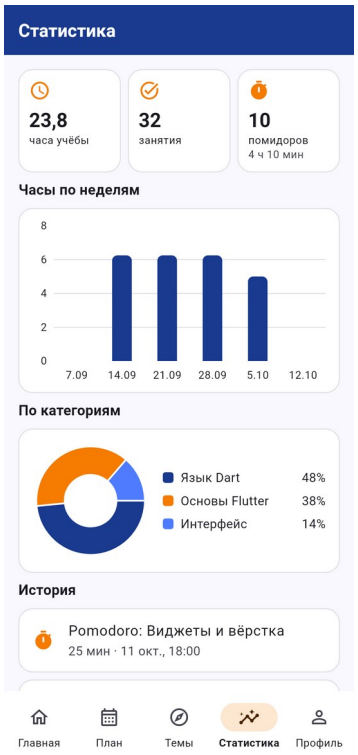


Рисунок 2.13 — Статистика

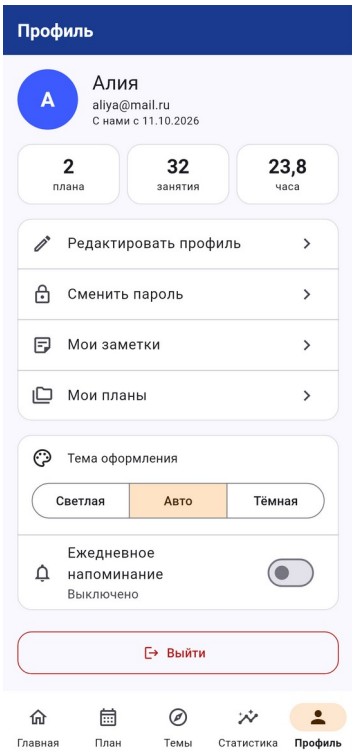


Рисунок 2.14 — Профиль

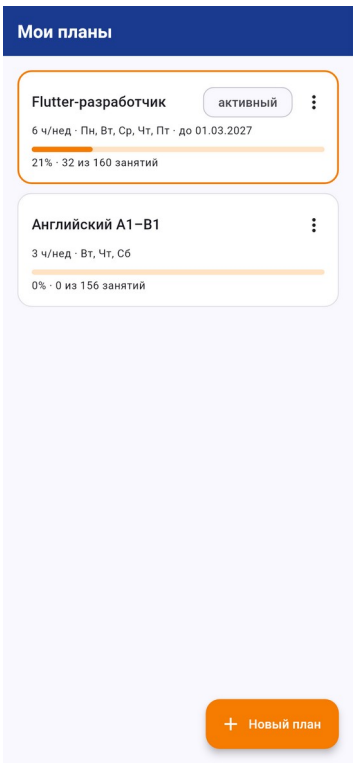


Рисунок 2.15 — «Мои планы»

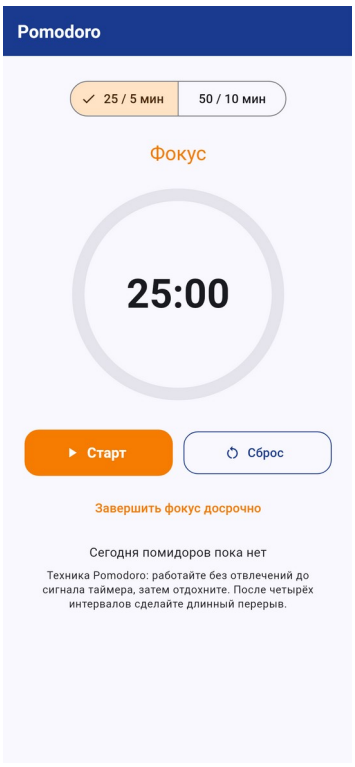


Рисунок 2.16 — Таймер Pomodoro

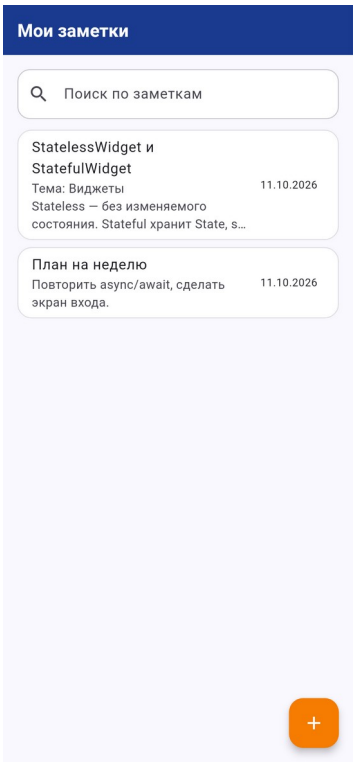


Рисунок 2.17 — Заметки

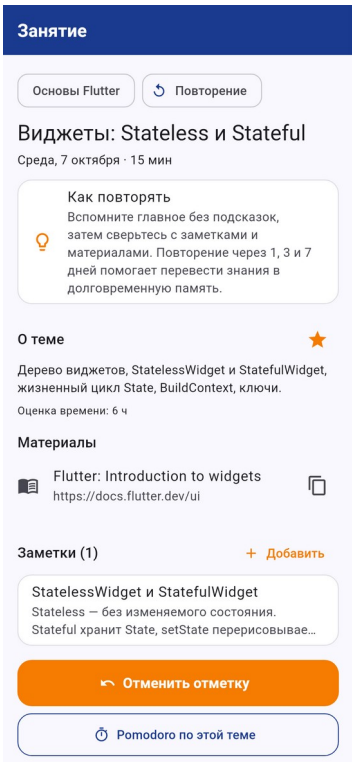


Рисунок 2.18 — Карточка занятия

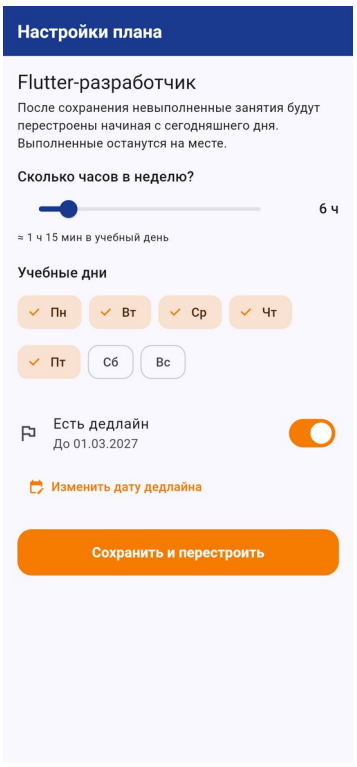


Рисунок 2.19 — Настройки плана

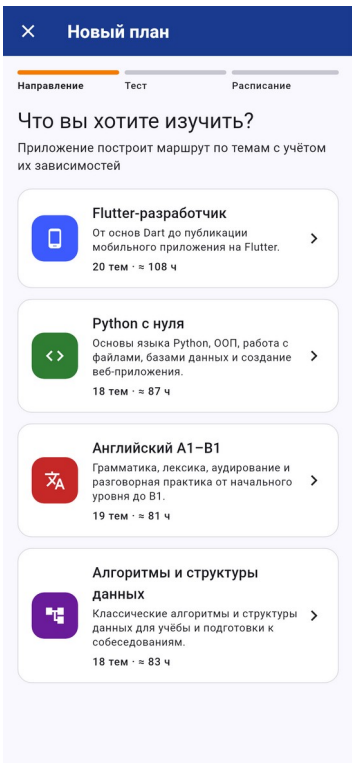


Рисунок 2.20 — Онбординг: выбор направления

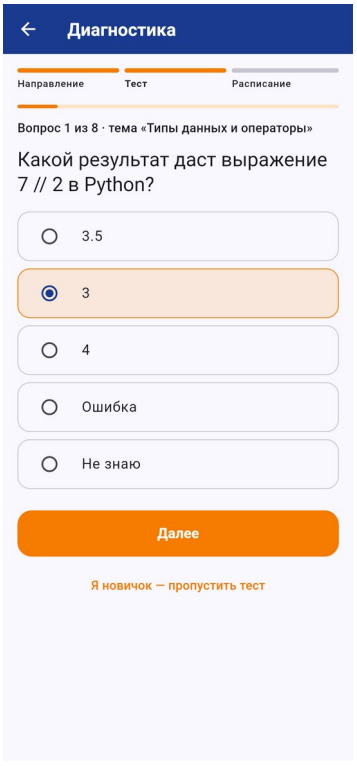


Рисунок 2.21 — Онбординг: диагностический тест

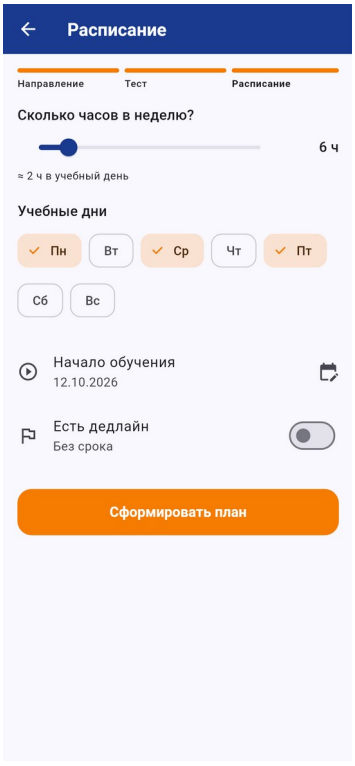


Рисунок 2.22 — Онбординг: расписание

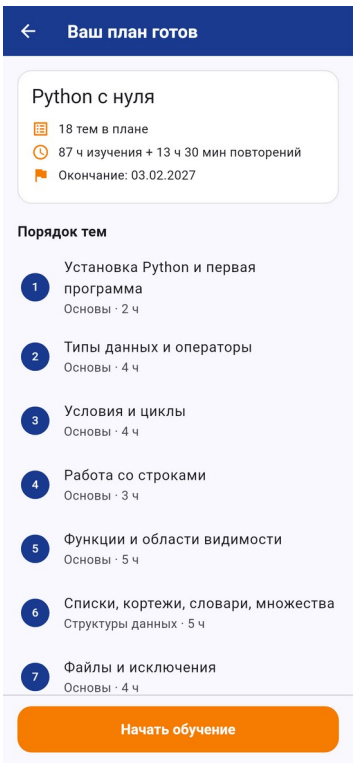


Рисунок 2.23 — Онбординг: предпросмотр плана

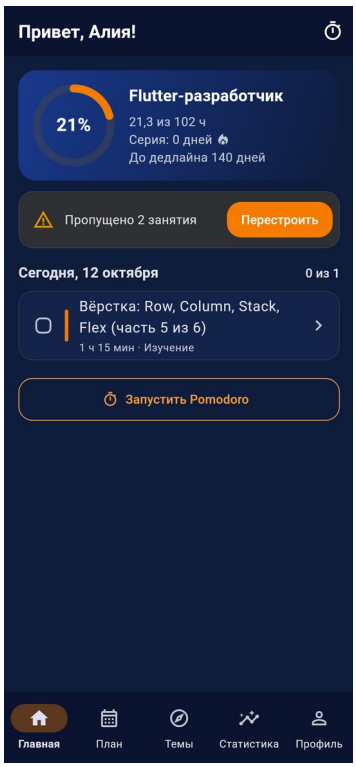


Рисунок 2.24 — Главная в тёмной теме

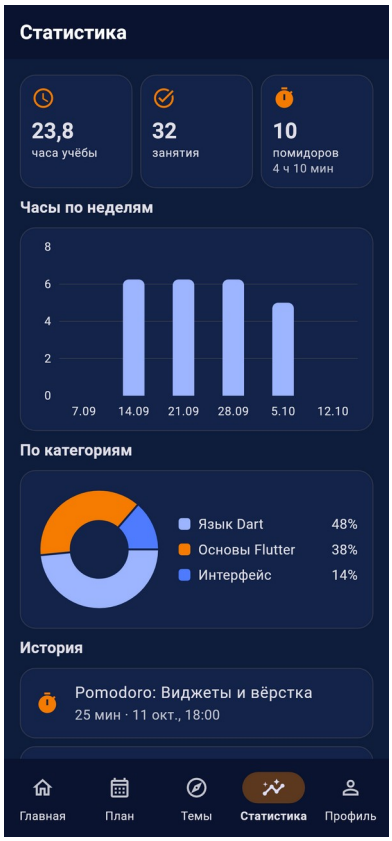


Рисунок 2.25 — Статистика в тёмной теме

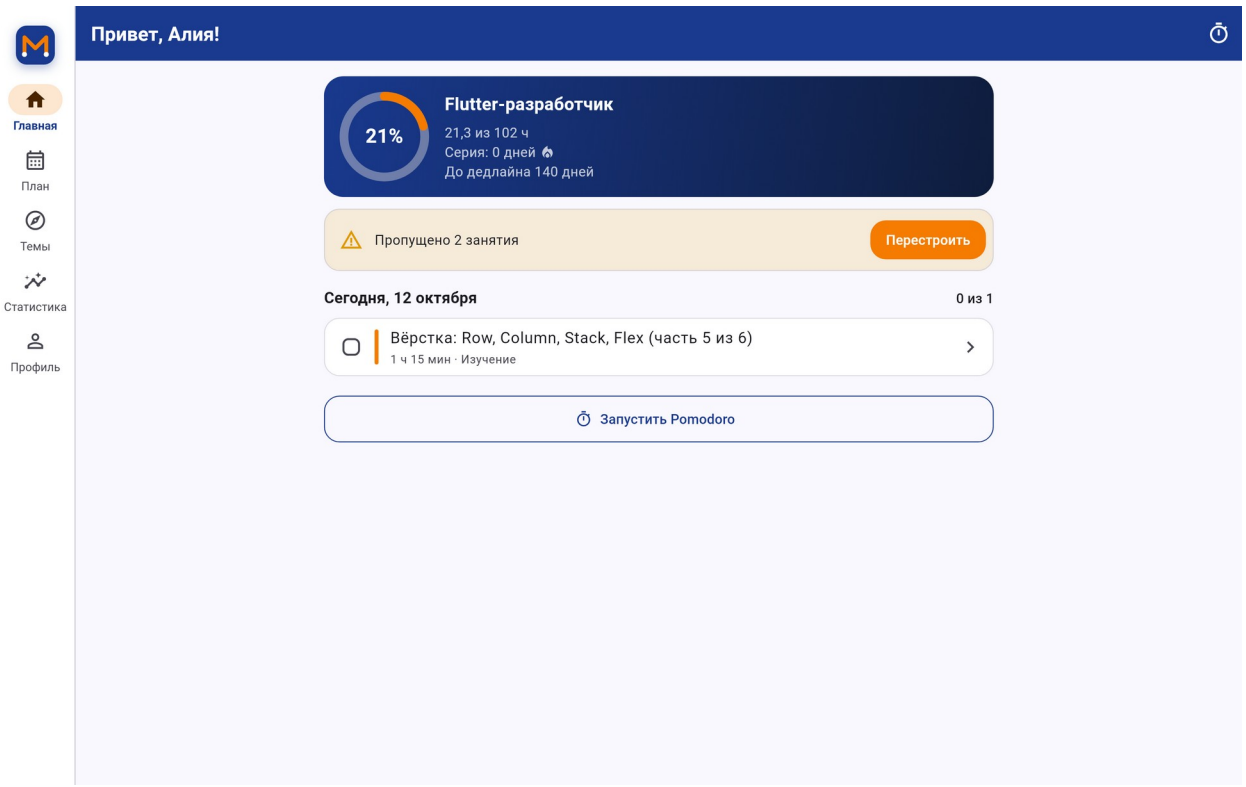


Рисунок 2.26 — Главная в окне компьютера

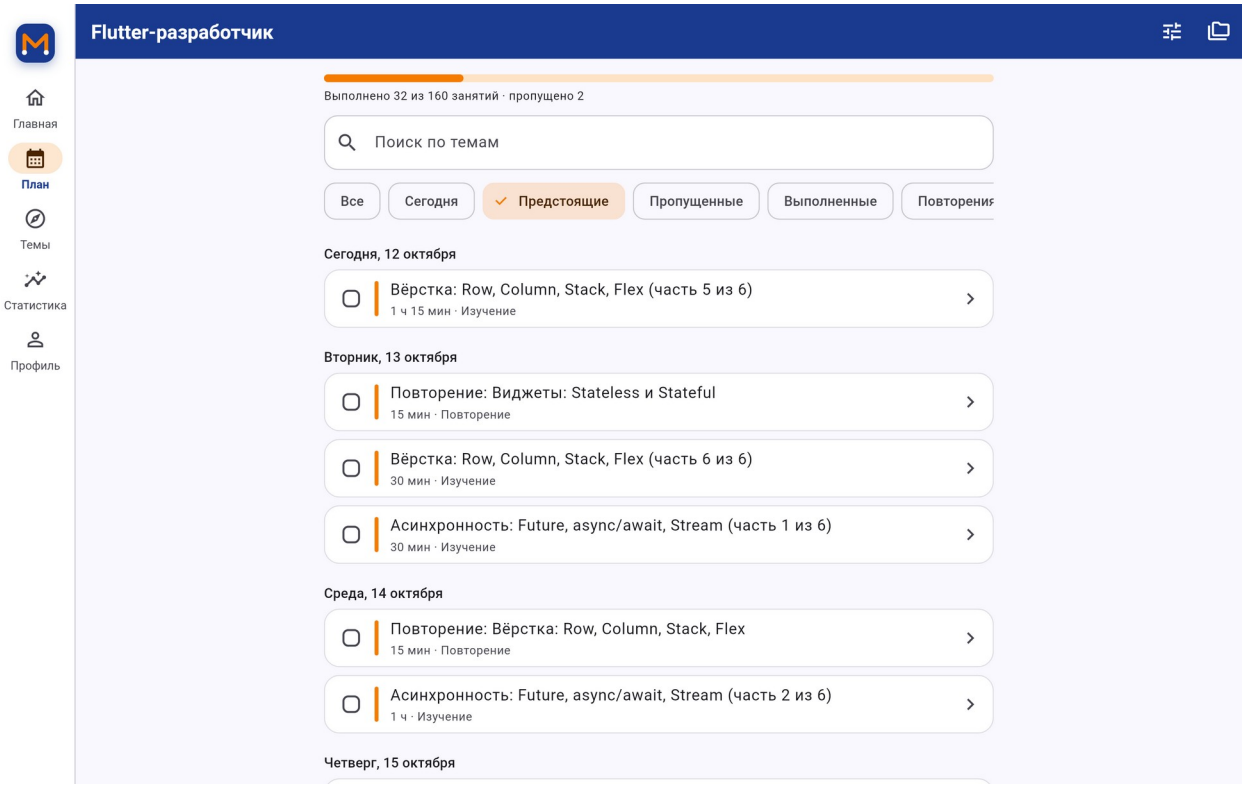


Рисунок 2.27 — План в окне компьютера

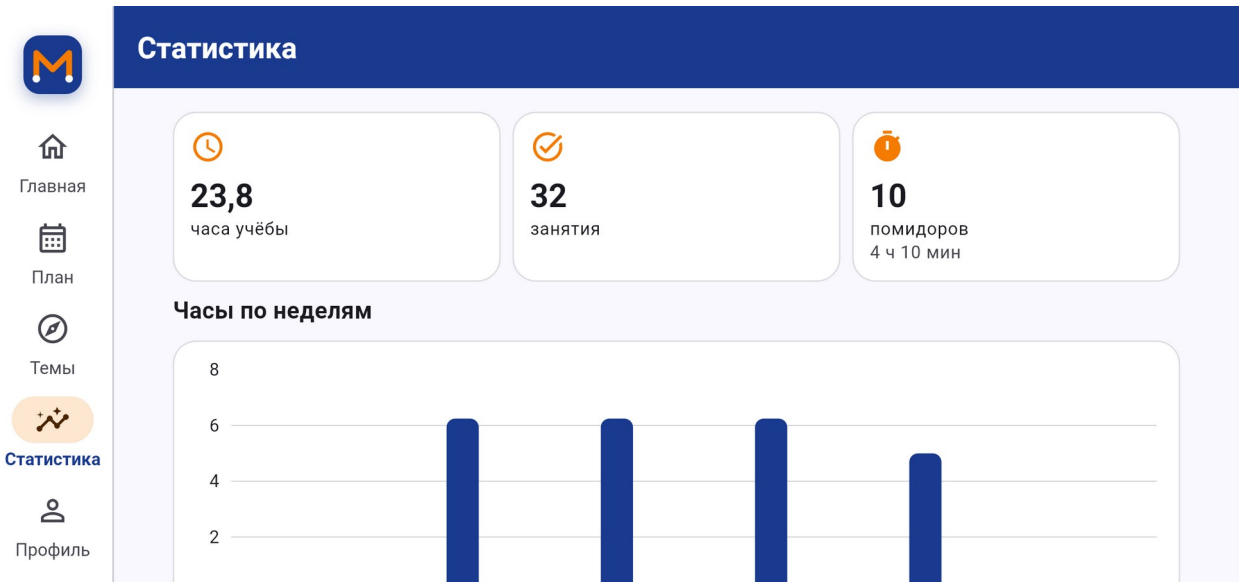


Рисунок 2.28 — Статистика в горизонтальной ориентации

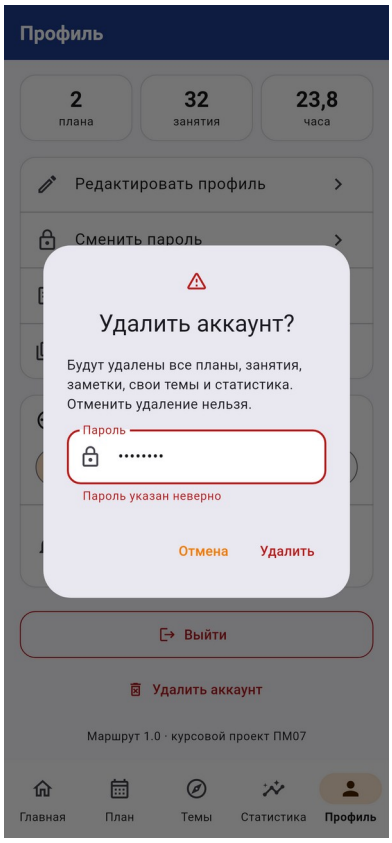


Рисунок 2.29 — Удаление аккаунта

2.6 Реализация основных модулей

2.6.1 Регистрация, вход и хранение паролей

Учётные записи хранятся в таблице `users`. При регистрации имя очищается от лишних пробелов, email приводится к нижнему регистру и

проверяется на уникальность. Пароль должен быть не короче 8 символов и содержать буквы и цифры.

В версии 1 семестра пароль хранился как SHA-256 от соли и пароля. SHA-256 вычисляется очень быстро, поэтому при копировании файла базы простые пароли можно подобрать перебором. Во 2 семестре хранение переведено на PBKDF2-HMAC-SHA256 [36], одну из функций, которые для паролей рекомендует OWASP [33]:

- соль — 16 случайных байт из криптостойкого генератора `Random.secure()`;
- функция HMAC-SHA256 применяется 100 000 раз, что во столько же раз замедляет перебор;
- результат хранится в виде `pbkdf2_sha256$100000$<хэш>`: число итераций записано в самом хэше, поэтому его можно увеличить в следующей версии, и старые записи продолжат проверяться;
- сравнение хэшей выполняется за постоянное время, чтобы время проверки не подсказывало, сколько символов совпало;
- расчёт выполняется в отдельном изоляте (`compute`), поэтому интерфейс не замирает на время входа.

Пользователи, зарегистрированные в версии 1 семестра, не теряют доступ: при входе приложение определяет формат хэша, проверяет пароль старым способом и сразу сохраняет новый хэш PBKDF2 с новой солью. Пароль известен приложению только в момент входа, поэтому пересчитать хэши заранее нельзя.

Корректность реализации PBKDF2 проверена по эталонным значениям, которые независимо посчитаны функцией `hashlib.pbkdf2_hmac` языка Python, в том числе для пароля на кириллице. Полный текст класса приведён в приложении Б.

При неверном пароле и при неизвестном email выводится одно и то же сообщение «Неверный email или пароль», чтобы по ответу нельзя было узнать, зарегистрирован ли адрес.

2.6.2 Сессия и выход

Если при входе отмечено «Запомнить меня», идентификатор пользователя сохраняется в `shared_preferences`. При следующем запуске Splash проверяет его: если пользователь существует, открывается главная, иначе — экран входа (это защищает от сессии удалённого аккаунта). При выходе идентификатор удаляется, ежедневное напоминание отменяется, а роутер открывает экран входа.

2.6.3 Удаление аккаунта

Удаление аккаунта необратимо, поэтому требует повторного ввода пароля. После проверки пароля удаляется строка пользователя, а планы, занятия, темы, заметки, избранное и история Pomodoro удаляются каскадно правилами внешних ключей. Затем выполняется выход.

2.6.4 Создание плана и работа с ним

Онбординг ведёт контроллер `OnboardingController`: он хранит выбранное направление, ответы теста и настройки расписания между шагами. На экране предпросмотра `PlanService.preview` рассчитывает план без сохранения, а `PlanService.createPlan` сохраняет темы и занятия в одной транзакции и делает новый план активным. Отметка занятия меняет одну строку таблицы `sessions`; главная, план и статистика получают изменение через потоки drift и обновляются сами.

2.6.5 Напоминания и уведомления

Ежедневное напоминание реализовано пакетом flutter_local_notifications [26]. Время напоминания хранится в профиле пользователя и пересчитывается в UTC, поэтому дополнительный пакет определения часового пояса не нужен. При окончании интервала Pomodoro показывается мгновенное уведомление. В Android-проекте объявлены приёмники плагина для показа запланированных уведомлений и их

восстановления после перезагрузки, разрешение `RECEIVE_BOOT_COMPLETED` и белый значок уведомлений. На Windows плагин не поддерживает ежедневное повторение без установщика MSIX, поэтому переключатель напоминания в настольной версии недоступен и подписан «Доступно в версии для Android и iOS»; мгновенные уведомления работают.

Сервис уведомлений скрыт за интерфейсом `ReminderService`. В тестах вместо него подставляется заглушка, которая записывает вызовы в журнал, поэтому тесты проверяют, что напоминание запланировано, не показывая настоящих уведомлений.

2.6.6 Обработка ошибок

Ошибки, понятные пользователю (занятый email, неверный пароль, пустое название), передаются исключением `AppException` с русским текстом и выводятся под полем формы или во всплывающем сообщении. Повреждённый файл каталога превращается в понятное сообщение вместо падения приложения. Ошибки платформы при работе с уведомлениями не прерывают выход из аккаунта.

2.7 Тестирование

Тестирование подробно описано в отдельном разделе (`docs/coursework/testing.md`): тест-кейсы, состав тестов и журнал найденных ошибок. Итоги приведены в таблице 2.6.

Таблица 2.6 — Итоги тестирования

Вид тестирования	**Объём**	**Результат**
Модульные тесты: алгоритм (`test/domain`)	32 теста	Пройдены
Модульные тесты: валидаторы, пароли, маршрутизация, каталог, форматирование	29 тестов	Пройдены
Тесты слоя данных на SQLite в памяти	18 тестов	Пройдены
Виджет-тесты экранов и сценариев	18 тестов	Пройдены
Интеграционный тест настоящего приложения (окно Linux)	1 сценарий	Пройден
Статический анализ `flutter analyze`	Весь проект	Замечаний нет
Ручная проверка скриншотов, настольной версии, настроек Android	25 скриншотов, 3 платформы	Найдено и исправлено 10 ошибок
Матрица снимков перед сдачей	280 снимков в 7 вариантах	Найдено и исправлено 10 ошибок
Тест-кейсы	36 автоматизированных, 6 ручных	См. `testing.md`

Всего найдено и исправлено 23 ошибки. Три из них нашли автоматические тесты: ошибку освобождения контроллера в диалоге переименования плана, слишком раннее ожидание экрана входа в интеграционном тесте и нажатие по старым координатам после прокрутки в самих виджет-тестах.

Тесты используют базу SQLite в оперативной памяти и фиксированную дату (понедельник 12.10.2026), поэтому результат не зависит от дня запуска и не затрагивает данные пользователя. Тест адаптивности открывает все экраны на трёх размерах окна; ошибка вёрстки (переполнение) автоматически роняет тест.

2.8 Сборка и развёртывание

Сборка автоматизирована в GitHub Actions [27] (файл `.github/workflows/build.yml`). При каждой отправке кода в репозиторий выполняются:

1. проверка форматирования кода, статический анализ и все модульные и виджет-тесты;

2. интеграционный тест настоящего приложения в Linux на виртуальном экране;
3. сборка релизного APK для Android (`flutter build apk --release`) [19];
4. сборка релизной версии для Windows (`flutter build windows --release`) [20].

Готовые файлы сохраняются как артефакты сборки. Когда на странице Releases репозитория публикуется новый релиз (например, `v1.0.0`), сборка запускается для него и прикладывает APK и архив для Windows к релизу, откуда их можно скачать без входа в GitHub.

Для установки на телефон нужно скачать файл `marshrut-release.apk`, открыть его и разрешить установку из этого источника. Релизный APK подписан отладочным ключом: для установки на свой телефон этого достаточно, для публикации в Google Play нужен собственный ключ подписи. Версия для Windows распространяется как архив: его нужно распаковать и запустить `marshrut.exe`. Для самостоятельной сборки достаточно установленного Flutter SDK: команды приведены в README.

Выводы по главе 2

1. Сформулированы функциональные и нефункциональные требования, все они реализованы в приложении.
2. Разработана архитектура из трёх слоёв. Алгоритм построения плана выделен в слой предметной области на чистом Dart и не зависит от интерфейса и базы данных.
3. Спроектирована база данных из восьми таблиц со связями и каскадным удалением.
4. Реализован алгоритм формирования персонального плана: исключение известных тем, топологическая сортировка Кана, раскладка по дням с лимитом времени, интервальные повторения, проверка дедлайна с подбором нужного числа часов и перепланирование.

5. Реализованы экраны приложения в стиле Material 3 с фирменными цветами, светлой и тёмной темами и адаптивной вёрсткой для телефона и компьютера.
6. По итогам анализа версии 1 семестра усилена защита паролей (PBKDF2), добавлено удаление аккаунта, настроены уведомления Android, сделаны иконка и релизная сборка.
7. Приложение проверено 97 модульными и виджет-тестами, интеграционным тестом и матрицей из 280 снимков экранов; найдено и исправлено 23 ошибки. Сборка APK и версии для Windows автоматизирована.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработано мобильное приложение «Маршрут», которое автоматически формирует персональный план обучения по выбранному направлению с учётом знаний пользователя, его времени и срока и перестраивает план при отставании. Цель работы достигнута, все поставленные задачи решены.

1. **Проанализирована предметная область.** Рассмотрены трудности самостоятельного обучения, методика распределённого повторения по кривой забывания Эббингауза и техника Pomodoro. Показано, что план обучения удобно описывать графом тем с зависимостями, а порядок изучения находить топологической сортировкой.
2. **Рассмотрены особенности мобильных приложений и принципы их разработки:** ограниченные ресурсы устройства, работа без сети, безопасность данных, адаптивность интерфейса, разделение кода на слои.
3. **Проведён анализ аналогов.** roadmap.sh, Duolingo, Stepik, Coursera, Notion и Todoist закрывают задачу частично; ни одно решение не объединяет граф тем, учёт знаний, расписание под время и срок пользователя и автоматическую перестройку плана.
4. **Обоснован выбор технологий:** Flutter и Dart, чистая архитектура, Riverpod, drift (SQLite), go_router, fl_chart, flutter_local_notifications, Material 3. Приложение работает без интернета и серверной части.
5. **Разработан алгоритм формирования плана.** Известные по тесту темы исключаются, остальные упорядочиваются алгоритмом Кана, раскладываются по учебным дням с дневным лимитом и дополняются повторениями через 1, 3 и 7 дней. Затем проверяется дедлайн, и при необходимости алгоритм подсказывает нужное число часов в неделю. Перепланирование сохраняет выполненное и раскладывает остаток

заново. Алгоритм выделен в слой на чистом Dart и покрыт 32 модульными тестами.

6. **Спроектирован интерфейс и реализованы экраны:** заставка, регистрация, вход, главная, пять вкладок навигации, профиль, онбординг с диагностическим тестом, план, темы, статистика, заметки, Pomodoro, несколько планов. Интерфейс выполнен в стиле Material 3 в тёмно-синих и оранжевых цветах, есть тёмная тема, вёрстка адаптирована для телефона в обеих ориентациях и для окна компьютера.
7. **Приложение протестировано:** 97 модульных и виджет-тестов, интеграционный тест настоящего приложения, 36 тест-кейсов, ручные проверки и матрица из 280 снимков экранов в светлой и тёмной теме, на разных размерах экрана и с крупным шрифтом. Найдено и исправлено 23 ошибки.
8. **Первая версия приложения проанализирована и доработана.** Хранение паролей переведено с однократного SHA-256 на PBKDF2-HMAC-SHA256 со 100 000 итераций с автоматическим пересчётом старых хэшей, добавлено удаление аккаунта, исправлена настройка уведомлений Android, сделаны иконка и название приложения. Сборка релизного APK для Android и версии для Windows автоматизирована в GitHub Actions вместе с запуском всех тестов.

Практическая значимость. Приложение можно использовать для реальной подготовки: в каталоге четыре направления (Flutter, Python, английский язык, алгоритмы), 75 тем с оценкой трудоёмкости, зависимостями и ссылками на материалы, а также 32 вопроса диагностического теста. Новое направление добавляется JSON-файлом без изменения кода.

Направления развития:

- синхронизация данных между устройствами и резервное копирование (облачная база или экспорт плана в файл);

- адаптация интервалов повторения к результатам пользователя, например по алгоритму SuperMemo;
- редактор зависимостей тем и импорт собственных направлений;
- публикация в Google Play с собственным ключом подписи;
- сборка и проверка версии для iOS.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

Список оформлен по ГОСТ 7.1-2003 «Библиографическая запись. Библиографическое описание», электронные ресурсы описаны по ГОСТ 7.82-2001. [ПРОВЕРИТЬ] Уточнить в методичке колледжа, какой стандарт требуется. Возможно, нужен ГОСТ Р 7.0.100-2018: тогда меняется только расстановка знаков, состав сведений остаётся тем же. Дата обращения ко всем электронным ресурсам: 11.10.2026. Перед сдачей проверить, открываются ли ссылки, и обновить дату.

Книги и статьи

1. Кормен, Т. Х. Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн ; пер. с англ. – 3-е изд. – М. : Вильямс, 2013. – 1328 с.
2. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; пер. с англ. – СПб. : Питер, 2018. – 352 с.
3. Cepeda, N. J. Distributed practice in verbal recall tasks: A review and quantitative synthesis / N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, D. Rohrer // Psychological Bulletin. – 2006. – Vol. 132, № 3. – P. 354–380.
4. Cirillo, F. The Pomodoro Technique: The Acclaimed Time-Management System That Has Transformed How We Work / F. Cirillo. – New York : Currency, 2018. – 144 p. [ПРОВЕРИТЬ число страниц]
5. Ebbinghaus, H. Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie / H. Ebbinghaus. – Leipzig : Duncker & Humblot, 1885. – 169 S. [ПРОВЕРИТЬ число страниц]
6. Kahn, A. B. Topological sorting of large networks / A. B. Kahn // Communications of the ACM. – 1962. – Vol. 5, № 11. – P. 558–562.
7. Napoli, M. L. Beginning Flutter: A Hands On Guide to App Development / M. L. Napoli. – Indianapolis : Wiley, 2019. – 528 p. [ПРОВЕРИТЬ число страниц]

8. Secure Hash Standard (SHS) : Federal Information Processing Standards Publication 180-4. – Gaithersburg : National Institute of Standards and Technology, 2015. – 36 p. – DOI 10.6028/NIST.FIPS.180-4. [ПРОВЕРИТЬ число страниц]
9. Windmill, E. Flutter in Action / E. Windmill. – Shelter Island : Manning, 2020. – 368 p. [ПРОВЕРИТЬ число страниц]

Электронные ресурсы

10. About SQLite [Электронный ресурс]. – Режим доступа: https://www.sqlite.org/about.html (дата обращения: 11.10.2026).
11. Adaptive icons [Электронный ресурс] // Android Developers. – Режим доступа: https://developer.android.com/develop/ui/views/launch/icon_design_adaptive (дата обращения: 11.10.2026).
12. Bloc library: documentation [Электронный ресурс]. – Режим доступа: https://bloclibrary.dev (дата обращения: 11.10.2026).
13. Coursera [Электронный ресурс] : онлайн-платформа курсов. – Режим доступа: https://www.coursera.org (дата обращения: 11.10.2026).
14. Dart documentation [Электронный ресурс] / Google. – Режим доступа: https://dart.dev/guides (дата обращения: 11.10.2026).
15. Drift: documentation [Электронный ресурс]. – Режим доступа: https://drift.simonbinder.eu (дата обращения: 11.10.2026).
16. Duolingo [Электронный ресурс] : сервис для изучения языков. – Режим доступа: https://www.duolingo.com (дата обращения: 11.10.2026).
17. Firebase documentation [Электронный ресурс] / Google. – Режим доступа: https://firebase.google.com/docs (дата обращения: 11.10.2026).
18. fl_chart [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа: https://pub.dev/packages/fl_chart (дата обращения: 11.10.2026).

19. Build and release an Android app [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/deployment/android (дата обращения: 11.10.2026).
20. Build and release a Windows desktop app [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/deployment/windows (дата обращения: 11.10.2026).
21. Guide to app architecture [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/app-architecture/guide (дата обращения: 11.10.2026).
22. Integration testing [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/testing/integration-tests (дата обращения: 11.10.2026).
23. List of state management approaches [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/data-and-backend/state-mgmt/options (дата обращения: 11.10.2026).
24. Supported deployment platforms [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/reference/supported-platforms (дата обращения: 11.10.2026).
25. Testing Flutter apps [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/testing/overview (дата обращения: 11.10.2026).
26. flutter_local_notifications [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа: https://pub.dev/packages/flutter_local_notifications (дата обращения: 11.10.2026).
27. GitHub Actions documentation [Электронный ресурс] / GitHub. – Режим доступа: https://docs.github.com/actions (дата обращения: 11.10.2026).

28. go_router [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа: https://pub.dev/packages/go_router (дата обращения: 11.10.2026).
29. Hive [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа: https://pub.dev/packages/hive (дата обращения: 11.10.2026).
30. Kotlin Multiplatform [Электронный ресурс] / JetBrains. – Режим доступа: https://www.jetbrains.com/kotlin-multiplatform/ (дата обращения: 11.10.2026).
31. Material Design 3 [Электронный ресурс] / Google. – Режим доступа: https://m3.material.io (дата обращения: 11.10.2026).
32. Notion [Электронный ресурс] : сервис заметок и задач. – Режим доступа: https://www.notion.so (дата обращения: 11.10.2026).
33. Password Storage Cheat Sheet [Электронный ресурс] // OWASP Cheat Sheet Series. – Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 11.10.2026).
34. Processes and app lifecycle [Электронный ресурс] // Android Developers. – Режим доступа: https://developer.android.com/guide/components/activities/process-lifecycle (дата обращения: 11.10.2026).
35. React Native [Электронный ресурс] / Meta. – Режим доступа: https://reactnative.dev (дата обращения: 11.10.2026).
36. RFC 8018. PKCS #5: Password-Based Cryptography Specification. Version 2.1 [Электронный ресурс] / K. Moriarty, B. Kaliski, A. Rusch. – IETF, 2017. – Режим доступа: https://www.rfc-editor.org/rfc/rfc8018 (дата обращения: 11.10.2026).
37. Riverpod: documentation [Электронный ресурс]. – Режим доступа: https://riverpod.dev (дата обращения: 11.10.2026).
38. roadmap.sh [Электронный ресурс] : карты изучения технологий. – Режим доступа: https://roadmap.sh (дата обращения: 11.10.2026).

39. Sound null safety [Электронный ресурс] // Dart documentation. – Режим доступа: https://dart.dev/null-safety (дата обращения: 11.10.2026).
40. Stepik [Электронный ресурс] : образовательная платформа. – Режим доступа: https://stepik.org (дата обращения: 11.10.2026).
41. Todoist [Электронный ресурс] : менеджер задач. – Режим доступа: https://todoist.com (дата обращения: 11.10.2026).
42. Understanding Google Play’s app account deletion requirements [Электронный ресурс] // Play Console Help. – Режим доступа: https://support.google.com/googleplay/android-developer/answer/13327111 (дата обращения: 11.10.2026). [ПРОВЕРИТЬ ссылку]
43. Use window size classes [Электронный ресурс] // Android Developers. – Режим доступа: https://developer.android.com/develop/ui/compose/layouts/adaptive/window-size-classes (дата обращения: 11.10.2026).

ПРИЛОЖЕНИЕ А. ТОПОЛОГИЧЕСКАЯ СОРТИРОВКА ТЕМ (АЛГОРИТМ КАНА)

Файл `lib/domain/planner/topological_sort.dart`.

```
import 'dart:collection';

import '../entities/topic.dart';

/// В графе тем обнаружен цикл, поэтому порядок изучения построить нельзя.
class CycleDetectedException implements Exception {
  CycleDetectedException(this.topicIds);

  /// Темы, которые остались неупорядоченными (входят в цикл или зависят от него).
  final List<String> topicIds;

  @override
  String toString() => 'В зависимостях тем есть цикл: ${topicIds.join(', ')}';
}

/// Топологическая сортировка тем алгоритмом Кана.
///
/// 1. Для каждой темы считаем входящую степень — число её зависимостей
///    внутри переданного набора. Зависимости, которых нет в наборе
///    (например, уже изученные темы), считаются выполненными.
/// 2. Темы с нулевой степенью помещаем в очередь с приоритетом.
/// 3. Извлекаем тему, добавляем в результат и уменьшаем степень у всех тем,
///    которые от неё зависят. Если степень стала нулевой — тема в очередь.
/// 4. Если в результат попали не все темы, в графе есть цикл.
///
/// Очередь упорядочена по уровню сложности, затем по исходной позиции темы,
/// поэтому при одинаковых входных данных результат всегда одинаковый.
/// Сложность — O((V + E) · log V) из-за очереди с приоритетом.
List<Topic> topologicalSort(List<Topic> topics) {
  final index = <String, int>{
    for (var i = 0; i < topics.length; i++) topics[i].id: i,
  };
  final inDegree = List<int>.filled(topics.length, 0);
  final dependents = List.generate(topics.length, (_) => <int>[]);

  for (var i = 0; i < topics.length; i++) {
    for (final prerequisite in topics[i].prerequisites.toSet()) {
      final from = index[prerequisite];
      if (from == null) continue; // зависимость уже изучена или вне плана
      dependents[from].add(i);
      inDegree[i]++;
    }
  }

  int compare(int a, int b) {
    final byLevel = topics[a].level.compareTo(topics[b].level);
    return byLevel != 0 ? byLevel : a.compareTo(b);
  }

  final queue = SplayTreeSet<int>(compare);
  for (var i = 0; i < topics.length; i++) {
    if (inDegree[i] == 0) queue.add(i);
  }

  final result = <Topic>[];
  while (queue.isNotEmpty) {
    final current = queue.first;
    queue.remove(current);
    result.add(topics[current]);
    for (final next in dependents[current]) {
      inDegree[next]--;
      if (inDegree[next] == 0) queue.add(next);
```

```
    }
  }

  if (result.length != topics.length) {
    final sorted = result.map((t) => t.id).toSet();
    throw CycleDetectedException(
      topics.where((t) => !sorted.contains(t.id)).map((t) => t.id).toList(),
    );
  }
  return result;
}
```

ПРИЛОЖЕНИЕ Б. ХЭШИРОВАНИЕ ПАРОЛЕЙ PBKDF2-HMAC-SHA256

Файл `lib/core/utils/password_hasher.dart`.

```
import 'dart:convert';
import 'dart:math';

import 'package:crypto/crypto.dart';
import 'package:flutter/foundation.dart' show compute;

/// Хэширование паролей алгоритмом PBKDF2-HMAC-SHA256 (RFC 8018).
///
/// Соль — 16 случайных байт из криптостойкого генератора [Random.secure].
/// Благодаря соли одинаковые пароли разных пользователей дают разные хэши,
/// а готовые таблицы хэшей («радужные таблицы») бесполезны. Многократное
/// повторение HMAC ([iterations] раз) замедляет подбор пароля перебором:
/// так рекомендует OWASP вместо однократного SHA-256.
///
/// Хэш хранится в виде `pbkdf2_sha256$<итерации>$<hex>`, поэтому число
/// итераций можно увеличить позже, не ломая старые записи. Хэши версии
/// 1 семестра (однократный SHA-256 от «соль + пароль») по-прежнему
/// проверяются и заменяются новыми при входе (см. [needsRehash]).
class PasswordHasher {
  PasswordHasher({Random? random, this.iterations = defaultIterations})
    : assert(iterations > 0),
      _random = random ?? Random.secure();

  /// Число итераций по умолчанию: заметно замедляет перебор и при этом
  /// не задерживает вход на телефоне дольше долей секунды.
  static const defaultIterations = 100000;

  static const _scheme = 'pbkdf2_sha256';

  final int iterations;
  final Random _random;

  String generateSalt([int length = 16]) {
    final bytes = List<int>.generate(length, (_) => _random.nextInt(256));
    return base64Url.encode(bytes);
  }

  /// Хэш пароля в текущем формате.
  String hash(String password, String salt) => _format(
    iterations,
    pbkdf2(utf8.encode(password), utf8.encode(salt), iterations),
  );

  /// [hash] в фоновом изоляте, чтобы интерфейс не замирал на время расчёта.
  Future<String> hashAsync(String password, String salt) {
    if (iterations < _isolateThreshold) {
      return Future.value(hash(password, salt));
    }
    final rounds = iterations;
    return compute(
      (_) => PasswordHasher(iterations: rounds).hash(password, salt),
      null,
    );
  }

  /// [verify] в фоновом изоляте.
  Future<bool> verifyAsync(String password, String salt, String stored) {
    final rounds = _iterationsOf(stored) ?? 1;
    if (rounds < _isolateThreshold) {
      return Future.value(verify(password, salt, stored));
    }
    return compute(
```

```
    (_) => PasswordHasher(iterations: 1).verify(password, salt, stored),
    null,
  );
}

/// При малом числе итераций запуск изолята дороже самого вычисления.
static const _isolateThreshold = 10000;

/// Хэш старого формата (версия 1 семестра): SHA-256 от «соль + пароль».
static String legacyHash(String password, String salt) =>
    sha256.convert(utf8.encode('$salt$password')).toString();

bool verify(String password, String salt, String stored) {
  final String actual;
  final rounds = _iterationsOf(stored);
  if (rounds != null) {
    actual = _format(
      rounds,
      pbkdf2(utf8.encode(password), utf8.encode(salt), rounds),
    );
  } else {
    actual = legacyHash(password, salt);
  }
  return _constantTimeEquals(actual, stored);
}

/// `true`, если хэш старого формата или посчитан с меньшим числом итераций.
bool needsRehash(String stored) {
  final rounds = _iterationsOf(stored);
  return rounds == null || rounds < iterations;
}

/// PBKDF2 с HMAC-SHA256. Для ключа длиной до 32 байт нужен один блок:
/// T = U1 ⊕ U2 ⊕ … ⊕ Uc, где U1 = HMAC(P, S ‖ 00000001), Uj = HMAC(P, Uj-1).
static List<int> pbkdf2(
  List<int> password,
  List<int> salt,
  int iterations, [
  int length = 32,
]) {
  final prf = Hmac(sha256, password);
  final blocks = (length + 31) ~/ 32;
  final result = <int>[];
  for (var block = 1; block <= blocks; block++) {
    var u = prf.convert([
      ...salt,
      block >> 24 & 0xff,
      block >> 16 & 0xff,
      block >> 8 & 0xff,
      block & 0xff,
    ]).bytes;
    final t = List<int>.of(u);
    for (var i = 1; i < iterations; i++) {
      u = prf.convert(u).bytes;
      for (var j = 0; j < t.length; j++) {
        t[j] ^= u[j];
      }
    }
    result.addAll(t);
  }
  return result.sublist(0, length);
}

static String _format(int rounds, List<int> key) =>
    '$_scheme\$$rounds\$${_hex(key)}';

static int? _iterationsOf(String stored) {
  final parts = stored.split(r'$');
  if (parts.length != 3 || parts[0] != _scheme) return null;
  final rounds = int.tryParse(parts[1]);
  return rounds != null && rounds > 0 ? rounds : null;
```

```
  }

  static String _hex(List<int> bytes) =>
      bytes.map((b) => b.toRadixString(16).padLeft(2, '0')).join();

  /// Сравнение без раннего выхода, чтобы время не зависело от совпадения.
  static bool _constantTimeEquals(String a, String b) {
    if (a.length != b.length) return false;
    var diff = 0;
    for (var i = 0; i < a.length; i++) {
      diff |= a.codeUnitAt(i) ^ b.codeUnitAt(i);
    }
    return diff == 0;
  }
}
```

ПРИЛОЖЕНИЕ В. РАСКЛАДКА ТЕМ ПО ДНЯМ И ИНТЕРВАЛЬНЫЕ ПОВТОРЕНИЯ

Файл `lib/domain/planner/scheduler.dart`.

```
import 'dart:collection';

import '../entities/schedule_settings.dart';
import '../entities/session.dart';
import '../entities/topic.dart';
import 'dates.dart';

/// Повторение, которое нужно провести не раньше [dueDate].
class PendingReview {
  const PendingReview({
    required this.topicId,
    required this.topicTitle,
    required this.category,
    required this.dueDate,
  });

  final String topicId;
  final String topicTitle;
  final String category;
  final DateTime dueDate;
}

/// Раскладывает упорядоченные темы по учебным дням.
///
/// Алгоритм моделирует календарь день за днём, начиная с даты старта:
/// 1. Невыбранные дни недели пропускаются.
/// 2. В учебный день доступно [ScheduleSettings.dailyLimitMinutes] минут.
/// 3. Сначала в день ставятся повторения, срок которых наступил
///    (по [reviewMinutes] минут каждое).
/// 4. Оставшееся время заполняется текущей темой. Тема дробится на части,
///    но не мельче [minChunkMinutes], если остаток темы больше этого порога.
/// 5. Когда тема закончена, планируются повторения через [reviewIntervals]
///    календарных дней (по кривой забывания Эббингауза: 1, 3, 7 дней).
///    Повторение, выпавшее на неучебный день, переносится на ближайший учебный.
class Scheduler {
  const Scheduler({
    this.reviewIntervals = const [1, 3, 7],
    this.reviewMinutes = 15,
    this.minChunkMinutes = 30,
    this.maxDays = 3650,
  });

  final List<int> reviewIntervals;
  final int reviewMinutes;
  final int minChunkMinutes;

  /// Защита от бесконечного цикла при некорректных настройках.
  final int maxDays;

  /// [remainingMinutes] позволяет передать остаток по теме, если часть уже
  /// изучена (используется при перепланировании). По умолчанию —
  /// полная длительность темы.
  /// [initialReviews] — повторения, перенесённые из прошлого расписания.
  List<PlannedSession> schedule(
    List<Topic> orderedTopics,
    ScheduleSettings settings, {
    Map<String, int> remainingMinutes = const {},
    List<PendingReview> initialReviews = const [],
  }) {
    final dailyLimit = settings.dailyLimitMinutes;
    if (settings.studyDays.isEmpty || dailyLimit <= 0) {
      throw ArgumentError('Нужно выбрать хотя бы один учебный день');
```

```
}

final work = Queue<_TopicWork>()
  ..addAll(
    orderedTopics
        .map((t) => _TopicWork(t, remainingMinutes[t.id] ?? t.minutes))
        .where((w) => w.remaining > 0),
  );
final reviews = [...initialReviews]
  ..sort((a, b) => a.dueDate.compareTo(b.dueDate));
final sessions = <PlannedSession>[];

var day = dateOnly(settings.startDate);
for (var i = 0; i < maxDays; i++, day = addDays(day, 1)) {
  if (work.isEmpty && reviews.isEmpty) break;
  if (!settings.studyDays.contains(day.weekday)) continue;

  var capacity = dailyLimit;

  // Шаг 3: повторения, срок которых наступил.
  final due = reviews.where((r) => !r.dueDate.isAfter(day)).toList();
  for (final review in due) {
    if (capacity < reviewMinutes) break;
    sessions.add(
      PlannedSession(
        topicId: review.topicId,
        topicTitle: review.topicTitle,
        category: review.category,
        date: day,
        minutes: reviewMinutes,
        kind: SessionKind.review,
      ),
    );
    capacity -= reviewMinutes;
    reviews.remove(review);
  }

  // Шаг 4: новые темы.
  while (work.isNotEmpty && capacity > 0) {
    final current = work.first;
    final chunk = capacity < current.remaining
        ? capacity
        : current.remaining;
    if (chunk < minChunkMinutes && chunk < current.remaining) break;

    sessions.add(
      PlannedSession(
        topicId: current.topic.id,
        topicTitle: current.topic.title,
        category: current.topic.category,
        date: day,
        minutes: chunk,
        kind: SessionKind.study,
      ),
    );
    capacity -= chunk;
    current.remaining -= chunk;

    if (current.remaining == 0) {
      work.removeFirst();
      // Шаг 5: повторения после завершения темы.
      for (final interval in reviewIntervals) {
        reviews.add(
          PendingReview(
            topicId: current.topic.id,
            topicTitle: current.topic.title,
            category: current.topic.category,
            dueDate: addDays(day, interval),
          ),
        );
      }
```

```
          reviews.sort((a, b) => a.dueDate.compareTo(b.dueDate));
        }
      }
    }

    return _numberParts(sessions);
  }

  /// Проставляет номер части для тем, разбитых на несколько занятий.
  List<PlannedSession> _numberParts(List<PlannedSession> sessions) {
    final totals = <String, int>{};
    for (final s in sessions) {
      if (s.kind == SessionKind.study) {
        totals[s.topicId] = (totals[s.topicId] ?? 0) + 1;
      }
    }
    final counters = <String, int>{};
    return [
      for (final s in sessions)
        if (s.kind == SessionKind.study)
          s.copyWith(
            part: counters[s.topicId] = (counters[s.topicId] ?? 0) + 1,
            totalParts: totals[s.topicId],
          )
        else
          s,
    ];
  }
}

class _TopicWork {
  _TopicWork(this.topic, this.remaining);

  final Topic topic;
  int remaining;
}
```

ПРИЛОЖЕНИЕ Г. ГЕНЕРАТОР ПЛАНА И ПРОВЕРКА ДЕДЛАЙНА

Файл `lib/domain/planner/plan_generator.dart`.

```
import '../entities/plan.dart';
import '../entities/schedule_settings.dart';
import '../entities/session.dart';
import '../entities/topic.dart';
import 'scheduler.dart';
import 'topological_sort.dart';

/// Генератор персонального плана обучения.
///
/// Конвейер из пяти шагов:
/// 1. Направление — граф тем с зависимостями (вход [topics]).
/// 2. Темы, известные по результатам теста ([knownTopicIds]), исключаются.
/// 3. Оставшиеся темы упорядочиваются топологической сортировкой (Кан).
/// 4. Темы раскладываются по учебным дням с лимитом часов ([Scheduler]).
/// 5. Проверяется дедлайн; если он не выполняется, подбирается минимальное
///    число часов в неделю, при котором план в него укладывается.
///    Интервальные повторения добавляет [Scheduler] на шаге 4.
class PlanGenerator {
  const PlanGenerator({this.scheduler = const Scheduler()});

  final Scheduler scheduler;

  /// Максимум часов в неделю, который имеет смысл предлагать (12 ч × 7 дней).
  static const double maxHoursPerWeek = 84;

  PlanResult generate({
    required List<Topic> topics,
    required ScheduleSettings settings,
    Set<String> knownTopicIds = const {},
  }) {
    // Шаг 2.
    final excluded = topics.where((t) => knownTopicIds.contains(t.id)).toList();
    final remaining = topics
        .where((t) => !knownTopicIds.contains(t.id))
        .toList();

    // Шаг 3.
    final ordered = topologicalSort(remaining);

    // Шаг 4.
    final sessions = scheduler.schedule(ordered, settings);
    final finish = lastStudyDate(sessions);

    // Шаг 5.
    final deadline = settings.deadline;
    final met = deadline == null || finish == null || !finish.isAfter(deadline);

    return PlanResult(
      orderedTopics: ordered,
      excludedTopics: excluded,
      sessions: sessions,
      finishDate: finish,
      deadlineMet: met,
      requiredHoursPerWeek: met
          ? null
          : requiredHoursPerWeek(ordered, settings),
    );
  }

  /// Подбирает минимальное число часов в неделю (с шагом 0,5 ч), при котором
  /// все темы будут изучены не позже дедлайна. Возвращает `null`, если даже
  /// [maxHoursPerWeek] не хватает.
  double? requiredHoursPerWeek(
    List<Topic> orderedTopics,
    ScheduleSettings settings, {
```

```
    Map<String, int> remainingMinutes = const {},
  }) {
    final deadline = settings.deadline;
    if (deadline == null) return null;

    for (
      var hours = settings.hoursPerWeek + 0.5;
      hours <= maxHoursPerWeek;
      hours += 0.5
    ) {
      final candidate = settings.copyWith(hoursPerWeek: hours);
      final sessions = scheduler.schedule(
        orderedTopics,
        candidate,
        remainingMinutes: remainingMinutes,
      );
      final finish = lastStudyDate(sessions);
      if (finish == null || !finish.isAfter(deadline)) return hours;
    }
    return null;
  }

  static DateTime? lastStudyDate(List<PlannedSession> sessions) {
    DateTime? last;
    for (final s in sessions) {
      if (s.kind != SessionKind.study) continue;
      if (last == null || s.date.isAfter(last)) last = s.date;
    }
    return last;
  }
}
```

ПРИЛОЖЕНИЕ Д. ПЕРЕПЛАНИРОВАНИЕ

Файл `lib/domain/planner/replanner.dart`.

```
import '../entities/schedule_settings.dart';
import '../entities/session.dart';
import '../entities/topic.dart';
import 'dates.dart';
import 'plan_generator.dart';
import 'scheduler.dart';

/// Результат перепланирования.
class ReplanResult {
  const ReplanResult({
    required this.sessions,
    required this.finishDate,
    required this.deadlineMet,
    this.requiredHoursPerWeek,
  });

  /// Новые невыполненные занятия, начиная с сегодняшнего дня.
  final List<PlannedSession> sessions;
  final DateTime? finishDate;
  final bool deadlineMet;
  final double? requiredHoursPerWeek;
}

/// Перестраивает план, если пользователь отстал или изменил расписание.
///
/// Выполненные занятия остаются на своих местах. Для каждой темы считается,
/// сколько минут уже изучено; остаток заново раскладывается начиная с
/// [today] теми же правилами, что и при генерации. Невыполненные повторения
/// по уже изученным темам переносятся на ближайшие учебные дни.
class Replanner {
  const Replanner({this.scheduler = const Scheduler()});

  final Scheduler scheduler;

  ReplanResult replan({
    required List<Topic> orderedTopics,
    required List<StudySession> sessions,
    required ScheduleSettings settings,
    required DateTime today,
  }) {
    final start = dateOnly(today);

    final doneMinutes = <String, int>{};
    for (final s in sessions) {
      if (s.isDone && s.kind == SessionKind.study) {
        doneMinutes[s.topicId] = (doneMinutes[s.topicId] ?? 0) + s.minutes;
      }
    }

    final remaining = <String, int>{
      for (final t in orderedTopics)
        t.id: (t.minutes - (doneMinutes[t.id] ?? 0)).clamp(0, t.minutes),
    };

    // Невыполненные повторения тем, которые уже изучены полностью.
    final pendingReviews = <PendingReview>[
      for (final s in sessions)
        if (!s.isDone &&
            s.kind == SessionKind.review &&
            remaining[s.topicId] == 0)
          PendingReview(
            topicId: s.topicId,
            topicTitle: s.topicTitle,
            category: s.category,
            dueDate: s.date.isBefore(start) ? start : s.date,
```

```
        ),
    ];

    final newSettings = settings.copyWith(startDate: start);
    final planned = scheduler.schedule(
      orderedTopics,
      newSettings,
      remainingMinutes: remaining,
      initialReviews: pendingReviews,
    );

    // Номера частей продолжают уже выполненные части темы.
    final doneParts = <String, int>{};
    for (final s in sessions) {
      if (s.isDone && s.kind == SessionKind.study) {
        doneParts[s.topicId] = (doneParts[s.topicId] ?? 0) + 1;
      }
    }
    final renumbered = [
      for (final s in planned)
        if (s.kind == SessionKind.study && (doneParts[s.topicId] ?? 0) > 0)
          s.copyWith(
            part: s.part + doneParts[s.topicId]!,
            totalParts: s.totalParts + doneParts[s.topicId]!,
          )
        else
          s,
    ];

    final finish = PlanGenerator.lastStudyDate(renumbered);
    final deadline = settings.deadline;
    final met = deadline == null || finish == null || !finish.isAfter(deadline);
    return ReplanResult(
      sessions: renumbered,
      finishDate: finish,
      deadlineMet: met,
      requiredHoursPerWeek: met
          ? null
          : PlanGenerator(scheduler: scheduler).requiredHoursPerWeek(
              orderedTopics,
              newSettings,
              remainingMinutes: remaining,
            ),
    );
  }
}
```

ПРИЛОЖЕНИЕ Е. ТАБЛИЦЫ БАЗЫ ДАННЫХ

Файл `lib/data/db/app_database.dart`.

```
import 'package:drift/drift.dart';

part 'app_database.g.dart';

// Таблицы базы данных SQLite. Код запросов генерирует drift:
//   dart run build_runner build
// Сгенерированный файл app_database.g.dart хранится в репозитории, поэтому
// для запуска приложения генерация не нужна.

/// Пользователи. Пароль хранится только в виде хэша PBKDF2-HMAC-SHA256 с солью
/// (записи версии 1 семестра с хэшем SHA-256 пересчитываются при входе).
@DataClassName('UserRow')
class Users extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get name => text().withLength(min: 2, max: 40)();
  TextColumn get email => text().unique()();
  TextColumn get passwordHash => text()();
  TextColumn get salt => text()();
  IntColumn get avatarColor => integer()();
  BoolColumn get reminderEnabled =>
      boolean().withDefault(const Constant(false))();
  IntColumn get reminderHour => integer().withDefault(const Constant(19))();
  IntColumn get reminderMinute => integer().withDefault(const Constant(0))();
  DateTimeColumn get createdAt => dateTime()();
}

/// Планы обучения. У пользователя может быть несколько планов, один активный.
@DataClassName('PlanRow')
class Plans extends Table {
  IntColumn get id => integer().autoIncrement()();
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();
  TextColumn get directionId => text()();
  TextColumn get title => text()();
  RealColumn get hoursPerWeek => real()();

  /// Учебные дни через запятую: «1,3,5» (1 — понедельник).
  TextColumn get studyDays => text()();
  DateTimeColumn get startDate => dateTime()();
  DateTimeColumn get deadline => dateTime().nullable()();

  /// Темы, исключённые по тесту, через запятую.
  TextColumn get knownTopicIds => text().withDefault(const Constant(''))();
  BoolColumn get isActive => boolean().withDefault(const Constant(true))();
  DateTimeColumn get createdAt => dateTime()();
}

/// Темы в составе плана — снимок, чтобы план не зависел от изменений каталога.
@DataClassName('PlanTopicRow')
class PlanTopics extends Table {
  IntColumn get id => integer().autoIncrement()();
  IntColumn get planId =>
      integer().references(Plans, #id, onDelete: KeyAction.cascade)();
  TextColumn get topicId => text()();
  TextColumn get title => text()();
  TextColumn get category => text()();
  IntColumn get minutes => integer()();
  IntColumn get level => integer()();
  TextColumn get prerequisites => text().withDefault(const Constant(''))();
  IntColumn get position => integer()();
}

/// Занятия плана (изучение темы или повторение) на конкретную дату.
@DataClassName('SessionRow')
class Sessions extends Table {
```

```
  IntColumn get id => integer().autoIncrement()();
  IntColumn get planId =>
      integer().references(Plans, #id, onDelete: KeyAction.cascade)();
  TextColumn get topicId => text()();
  TextColumn get topicTitle => text()();
  TextColumn get category => text()();
  DateTimeColumn get date => dateTime()();
  IntColumn get minutes => integer()();

  /// «study» или «review».
  TextColumn get kind => text()();
  IntColumn get part => integer().withDefault(const Constant(1))();
  IntColumn get totalParts => integer().withDefault(const Constant(1))();
  BoolColumn get isDone => boolean().withDefault(const Constant(false))();
  DateTimeColumn get completedAt => dateTime().nullable()();
}

/// Собственные темы пользователя.
@DataClassName('CustomTopicRow')
class CustomTopics extends Table {
  IntColumn get id => integer().autoIncrement()();
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();
  TextColumn get directionId => text()();
  TextColumn get title => text()();
  TextColumn get description => text().withDefault(const Constant(''))();
  IntColumn get minutes => integer()();
  TextColumn get category => text()();
  TextColumn get prerequisites => text().withDefault(const Constant(''))();
  DateTimeColumn get createdAt => dateTime()();
}

/// Заметки пользователя.
@DataClassName('NoteRow')
class Notes extends Table {
  IntColumn get id => integer().autoIncrement()();
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();
  TextColumn get title => text()();
  TextColumn get body => text()();
  TextColumn get topicId => text().nullable()();
  TextColumn get topicTitle => text().nullable()();
  DateTimeColumn get createdAt => dateTime()();
  DateTimeColumn get updatedAt => dateTime()();
}

/// Избранные темы.
@DataClassName('FavoriteRow')
class Favorites extends Table {
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();
  TextColumn get topicId => text()();
  DateTimeColumn get createdAt => dateTime()();

  @override
  Set<Column> get primaryKey => {userId, topicId};
}

/// Завершённые интервалы Pomodoro.
@DataClassName('PomodoroLogRow')
class PomodoroLogs extends Table {
  IntColumn get id => integer().autoIncrement()();
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();
  IntColumn get minutes => integer()();
  TextColumn get topicTitle => text().nullable()();
  DateTimeColumn get completedAt => dateTime()();
}

@DriftDatabase(
  tables: [
```

```
    Users,
    Plans,
    PlanTopics,
    Sessions,
    CustomTopics,
    Notes,
    Favorites,
    PomodoroLogs,
  ],
)
class AppDatabase extends _$AppDatabase {
  AppDatabase(super.executor);

  @override
  int get schemaVersion => 1;

  @override
  MigrationStrategy get migration => MigrationStrategy(
    onCreate: (m) => m.createAll(),
    beforeOpen: (details) async {
      // Включаем проверку внешних ключей (каскадное удаление планов и занятий).
      await customStatement('PRAGMA foreign_keys = ON');
    },
  );
}
```

ПРИЛОЖЕНИЕ Ж. ПРАВИЛО ДОСТУПА К ЭКРАНАМ И МАРШРУТЫ

Файл `lib/core/router/app_router.dart`.

```
/// Маршруты, доступные без входа в систему.
const publicRoutes = {'/login', '/register'};

/// Правило доступа к экранам (вынесено в функцию, чтобы его можно было
/// проверить юнит-тестом без запуска интерфейса).
///
/// * пока сессия не проверена — показываем только Splash;
/// * неавторизованный пользователь видит только вход и регистрацию;
/// * авторизованного пользователя с экранов входа и Splash ведём на главную.
@visibleForTesting
String? authRedirect(AuthStatus status, String location) {
  if (status == AuthStatus.unknown) {
    return location == '/splash' ? null : '/splash';
  }
  final loggedIn = status == AuthStatus.authenticated;
  final isPublic = publicRoutes.contains(location);
  if (!loggedIn) return isPublic ? null : '/login';
  if (isPublic || location == '/splash') return '/home';
  return null;
}

final _rootKey = GlobalKey<NavigatorState>(debugLabel: 'root');

final routerProvider = Provider<GoRouter>((ref) {
  // Роутер пересчитывает redirect каждый раз, когда меняется состояние входа.
  final refresh = ValueNotifier<int>(0);
  ref.listen(authControllerProvider, (_, _) => refresh.value++);
  ref.onDispose(refresh.dispose);

  final router = GoRouter(
    navigatorKey: _rootKey,
    initialLocation: '/splash',
    refreshListenable: refresh,
    redirect: (context, state) => authRedirect(
      ref.read(authControllerProvider).status,
      state.matchedLocation,
    ),
    routes: [
      GoRoute(path: '/splash', builder: (_, _) => const SplashScreen()),
      GoRoute(
        path: '/login',
        builder: (_, state) => LoginScreen(
          initialEmail: state.uri.queryParameters['email'],
          justRegistered: state.uri.queryParameters['registered'] == '1',
        ),
      ),
      GoRoute(path: '/register', builder: (_, _) => const RegisterScreen()),

      // Онбординг: создание нового плана.
      GoRoute(
        path: '/onboarding',
        builder: (_, _) => const DirectionScreen(),
        routes: [
          GoRoute(path: 'test', builder: (_, _) => const DiagnosticScreen()),
          GoRoute(path: 'schedule', builder: (_, _) => const ScheduleScreen()),
          GoRoute(path: 'preview', builder: (_, _) => const PreviewScreen()),
        ],
      ),

      // Основные вкладки с нижней навигацией.
      StatefulShellRoute.indexedStack(
        builder: (_, _, shell) => MainShell(navigationShell: shell),
```

```
  branches: [
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/home', builder: (_, _) => const HomeScreen()),
      ],
    ),
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/plan', builder: (_, _) => const PlanScreen()),
      ],
    ),
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/topics', builder: (_, _) => const TopicsScreen()),
      ],
    ),
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/stats', builder: (_, _) => const StatsScreen()),
      ],
    ),
    StatefulShellBranch(
      routes: [
        GoRoute(
          path: '/profile',
          builder: (_, _) => const ProfileScreen(),
        ),
      ],
    ),
  ],
),

// Экраны поверх вкладок.
GoRoute(
  path: '/session/:id',
  builder: (_, state) =>
      SessionScreen(sessionId: int.parse(state.pathParameters['id']!)),
),
GoRoute(path: '/plans', builder: (_, _) => const PlansScreen()),
GoRoute(
  path: '/plan/settings',
  builder: (_, _) => const PlanSettingsScreen(),
),
GoRoute(
  path: '/pomodoro',
  builder: (_, state) =>
      PomodoroScreen(topicTitle: state.uri.queryParameters['topic']),
),
GoRoute(
  path: '/topic/:id',
  builder: (_, state) =>
      TopicScreen(topicId: state.pathParameters['id']!),
),
GoRoute(
  path: '/topics/custom/new',
  builder: (_, state) => CustomTopicScreen(
    directionId: state.uri.queryParameters['direction'],
  ),
),
GoRoute(
  path: '/topics/custom/:id',
  builder: (_, state) =>
      CustomTopicScreen(topicId: state.pathParameters['id']),
),
GoRoute(path: '/notes', builder: (_, _) => const NotesScreen()),
GoRoute(
  path: '/notes/edit',
  builder: (_, state) {
    final q = state.uri.queryParameters;
    return NoteEditScreen(
      // Новый ключ для каждой заметки: иначе при переходе между
```

```
            // заметками сохранялось бы состояние предыдущей.
            key: ValueKey(state.uri.toString()),
            noteId: int.tryParse(q['id'] ?? ''),
            topicId: q['topicId'],
            topicTitle: q['topicTitle'],
          );
        },
      ),
      GoRoute(
        path: '/profile/edit',
        builder: (_, _) => const EditProfileScreen(),
      ),
      GoRoute(
        path: '/profile/password',
        builder: (_, _) => const ChangePasswordScreen(),
      ),
    ],
  );
  ref.onDispose(router.dispose);
  return router;
});
```

ПРИЛОЖЕНИЕ И. МОДУЛЬНЫЕ ТЕСТЫ ТОПОЛОГИЧЕСКОЙ СОРТИРОВКИ

Файл `test/domain/topological_sort_test.dart`.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:marshrut/domain/entities/topic.dart';
import 'package:marshrut/domain/planner/topological_sort.dart';

Topic t(String id, {List<String> deps = const [], int level = 1}) => Topic(
  id: id,
  title: id,
  minutes: 60,
  category: 'Тест',
  level: level,
  prerequisites: deps,
);

List<String> ids(List<Topic> topics) => topics.map((t) => t.id).toList();

void main() {
  group('topologicalSort (алгоритм Кана)', () {
    test('каждая тема идёт после всех своих зависимостей', () {
      final topics = [
        t('riverpod', deps: ['widgets', 'async']),
        t('widgets', deps: ['dart']),
        t('async', deps: ['dart']),
        t('dart'),
      ];

      final order = ids(topologicalSort(topics));

      expect(order.first, 'dart');
      expect(order.last, 'riverpod');
      for (final topic in topics) {
        for (final dep in topic.prerequisites) {
          expect(
            order.indexOf(dep) < order.indexOf(topic.id),
            isTrue,
            reason: '$dep должна идти раньше ${topic.id}',
          );
        }
      }
    });

    test(
      'независимые темы упорядочены по уровню, затем по исходной позиции',
      () {
        final topics = [
          t('hard', level: 3),
          t('easy_b', level: 1),
          t('medium', level: 2),
          t('easy_a', level: 1),
        ];

        expect(ids(topologicalSort(topics)), [
          'easy_b',
          'easy_a',
          'medium',
          'hard',
        ]);
      },
    );

    test('зависимость, которой нет в наборе, считается выполненной', () {
      final topics = [
        t('b', deps: ['known_topic']),
```

```
        t('a'),
      ];

      expect(ids(topologicalSort(topics)), ['b', 'a']);
    });

    test('повторяющаяся зависимость не ломает подсчёт степени', () {
      final topics = [
        t('b', deps: ['a', 'a']),
        t('a'),
      ];

      expect(ids(topologicalSort(topics)), ['a', 'b']);
    });

    test('пустой список даёт пустой результат', () {
      expect(topologicalSort(const []), isEmpty);
    });

    test('цикл в графе обнаруживается', () {
      final topics = [
        t('a', deps: ['c']),
        t('b', deps: ['a']),
        t('c', deps: ['b']),
        t('free'),
      ];

      expect(
        () => topologicalSort(topics),
        throwsA(
          isA<CycleDetectedException>().having(
            (e) => e.topicIds,
            'topicIds',
            unorderedEquals(['a', 'b', 'c']),
          ),
        ),
      );
    });
  });
}
```